



Java Básico

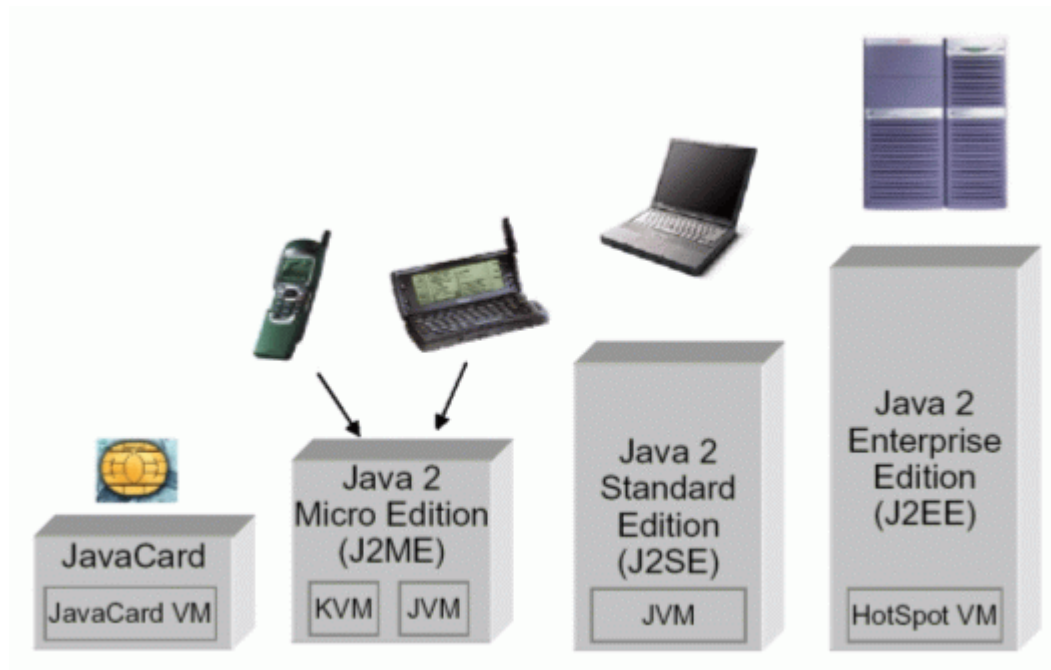
Programando em Java

(Com alterações introduzidas por Marcelo Ladeira e Thelmo Enoque)

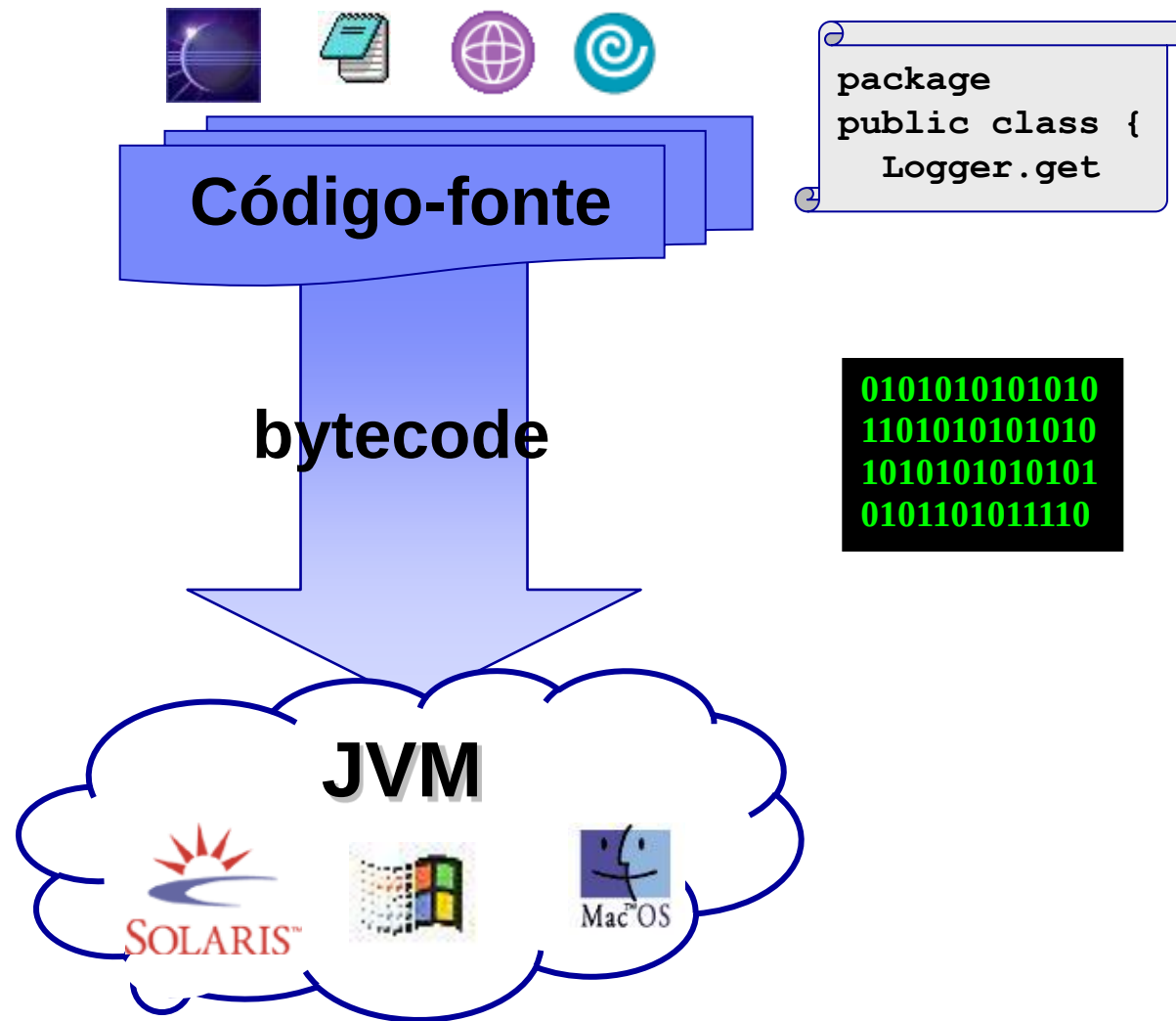
Java Características

- Simples
- Orientada a Objetos
- Distribuída
- Suporte a Concorrência
- Dinâmica
- Independente de Plataforma
- Portável
- Alta Performance
- Robusta
- Segura

Java em Toda Parte



Entendo o Funcionamento Java



Conceitos Básicos

- ByteCode

- Código intermediário (*.class), entre o código fonte (*.java) e o código de máquina.

- Compilador

- Responsável pela geração do bytecode (*.class) através da tradução do código fonte.

- JVM

- Máquina virtual do Java, responsável pela leitura do bytecode, tradução para a linguagem de máquina e execução do programa.

- Classpath

- Conjunto de caminhos especificado para a JVM encontrar as classes necessárias para a execução do programa.



| Competência Java

Orientação a Objetos

| Programando em Java

| Aprendendo Java

© 2006 IBM Corporation

Quinta Geração de Linguagens de Programação

- Primeira Geração: linguagem de máquina
- Segunda Geração: linguagem de montagem (assembly)
- Terceira Geração: linguagem de alto nível
- Quarta Geração: linguagens para geração de aplicações
- Geração Orientada a Objetos: **linguagens voltadas para reuso e manutenção.**

A Quinta Geração enfrenta o desafio de efetuar a manutenção e o reuso das milhares de aplicações desenvolvidas pelas gerações anteriores e atual.

Programação Orientada a Objetos: POO

- Com linguagens procedurais, o programador fornece um algoritmo para solucionar o problema a resolver.
- Com linguagens declarativas, o programador descreve uma solução para o problema e a VM provê o suporte para transformá-la em comandos executáveis pelo computador.
- Com POO, o programador descreve o problema em termos do próprio problema, ao invés de descrevê-lo em termos de um algoritmo que o computador vai rodar.
 - Abordagem híbrido entre a procedural e a funcional?

Orientação a Objetos

- Em POO os objetos são abstrações de elementos que pertencem ao modelo do problema.
- A idéia é que o programa possa se adaptar ao domínio do problema a medida que ele incorpora esses mesmos objetos. Assim, lendo o código, estaremos lendo palavras que também expressam o problema.

Alguns projetistas de linguagem entendem que POO não é adequada para resolver todos os tipos de problemas e defendem a combinação de várias abordagens nas linguagens de *multiparadigmas*.

Orientação a Objetos

Todas as coisas são objetos!

- Pense em um objeto como uma variável que além de armazenar dados, realiza as operações com esses dados e tem um comportamento definido.
- Conceitualmente, todas as coisas do domínio de seu problema podem ser realizadas em objetos (casas, interruptores, cachorros, serviços, etc...)

Orientação a Objetos

Programas

- Programas são conjuntos de objetos dizendo uns aos outros o que fazer através de mensagens.
- Para pedir que um determinado objeto faça alguma tarefa você deve enviar-lhe uma mensagem.
- Mais concretamente, as mensagens são chamadas à função, ou em jargão de POO, invocações de método.

Orientação a Objetos

Objetos derivados de outros objetos

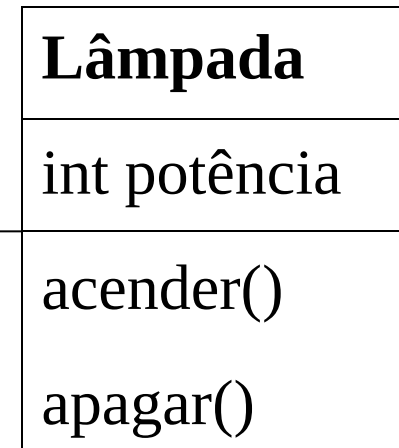
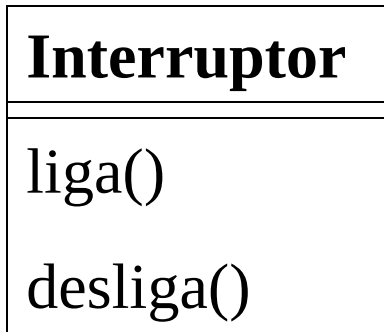
- Podemos criar objetos a partir da especialização de objetos mais genéricos.
- Objetos podem conter dentro de si outros objetos.
- Objetos podem substituir métodos de outros objetos.
- Dessa forma podemos compor soluções complexas combinando objetos mais simples.

Orientação a Objetos

Todo objeto tem um tipo

- Todo objeto é instância de uma classe.
 - A característica mais singular em uma classe é que tipo de mensagem que ela pode receber.
 - Todos os objetos de uma classe podem receber as mesmas mensagens.
 - Todo objeto que é um círculo é também uma figura geométrica. Logo o círculo pode receber todas as mensagens destinadas a uma figura geométrica.
 - O código escrito para lidar com figuras geométricas serve para todos os seus tipos derivados.

Orientação a Objetos:



```
class Interruptor {  
    Lampada lp = new Lampada();  
    public void liga() {  
        lp.acender()  
    }  
    public void desliga() {  
        lp.apagar()  
    }  
}
```

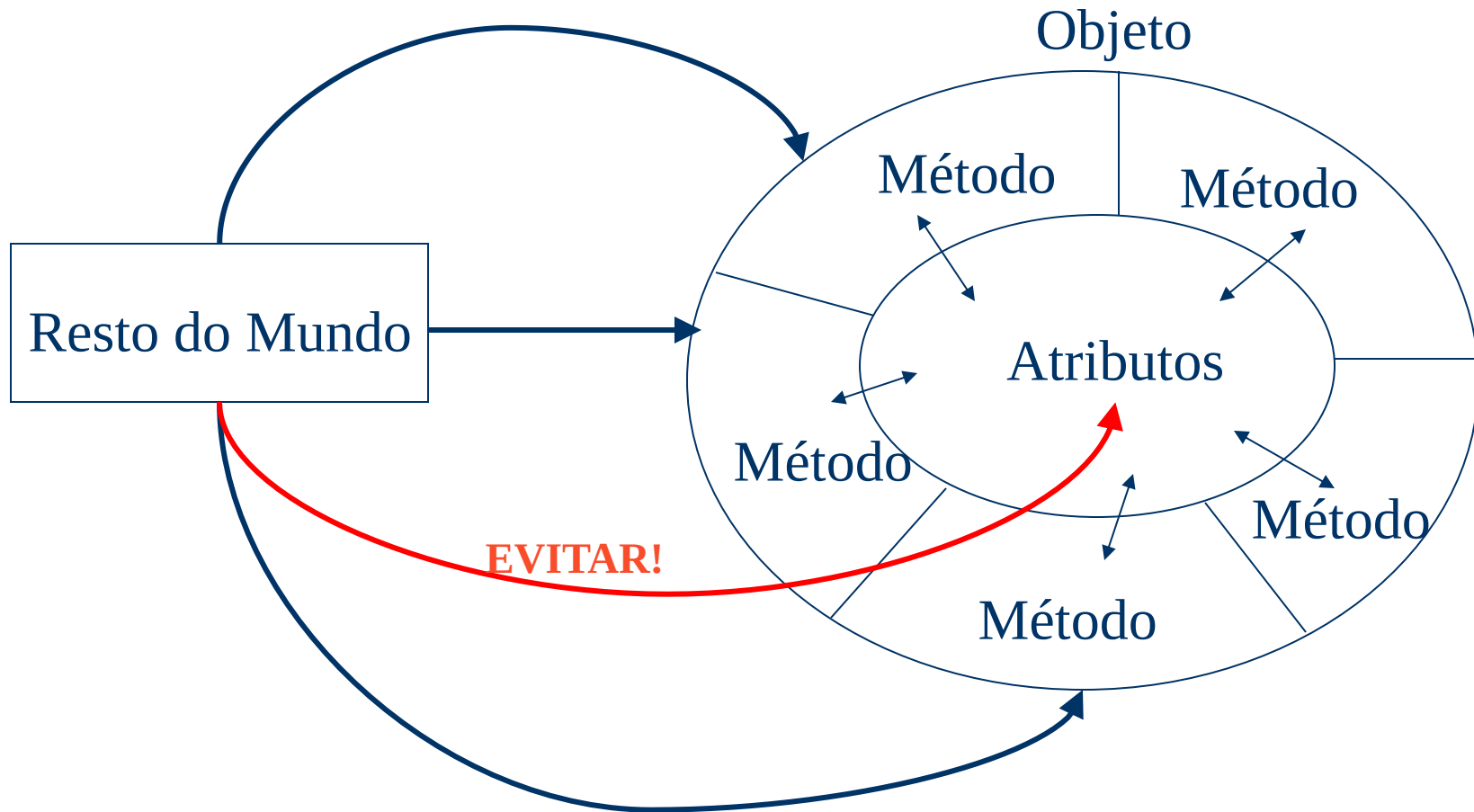
```
class Lampada {  
    int potencia = 60;  
    public void  
    acender() { /*...*/ }  
    public void  
    apagar() { /*...*/ }  
}
```

Atributos e métodos



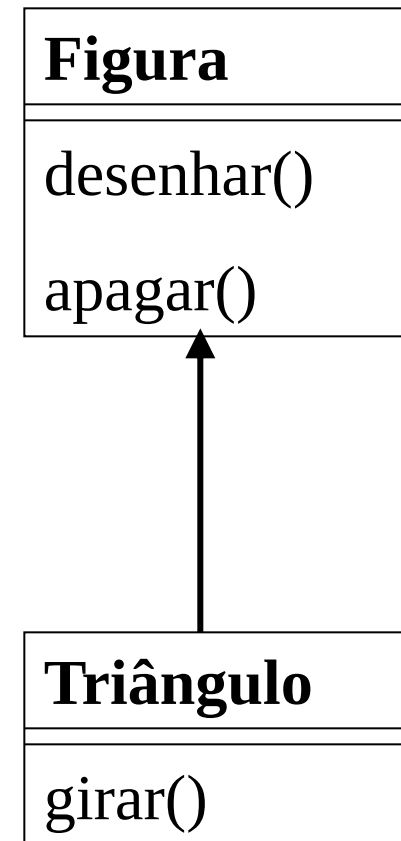
- Atributos representam uma propriedade, comum a todas as instâncias da classe que se está modelando.
 - toda lâmpada tem uma potência.
- Métodos são as ações que uma classe pode fazer, também são os agentes de todas as mensagens.
 - um interruptor pode ligar e desligar.

Mensagens e Visibilidade



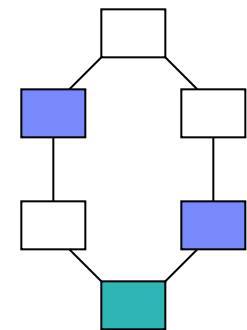
Herança

- Herança é o mecanismo de incorporar automaticamente métodos e atributos de uma classe definida em suas classes derivadas.
- Você pode definir métodos da classe sendo criada, assim como operações para especializar, substituir, ou estender a funcionalidade dos métodos da superclasse.
 - Uma classe herda apenas de uma superclasse.



Herança múltipla

- Em Java não há herança múltipla:
 - uma classe só pode ter uma superclasse.
 - existe uma perda de flexibilidade compensada por um ganho de legibilidade do código.
 - Evita-se situações dúbias como saber de qual superclasse se herda atributos de mesmo nome que pertencem a mais de uma superclasse.
 - Menor distância inferencial?

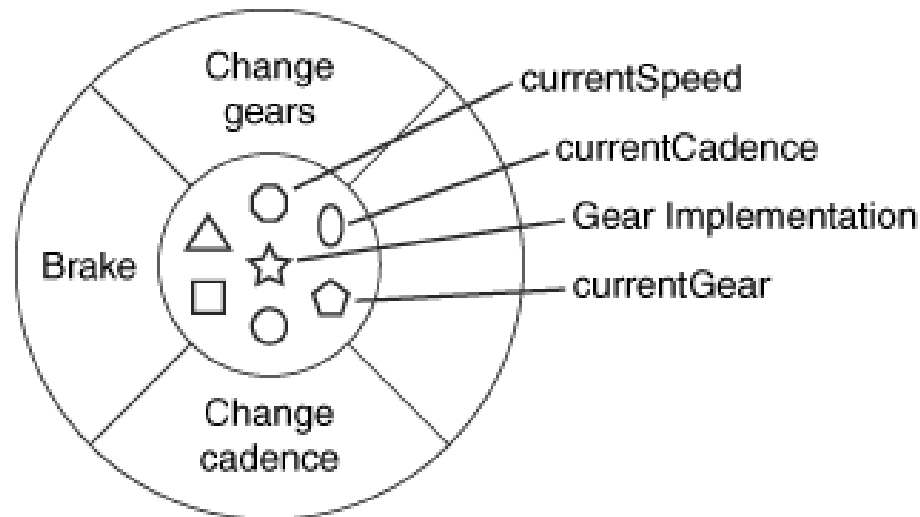


Conceitos Básicos

- Conceitos chaves da programação Orientada a Objetos:
 - **Classe**
 - Um modelo que descreve um objeto
 - **Objetos**
 - Representam entidades existentes no mundo real

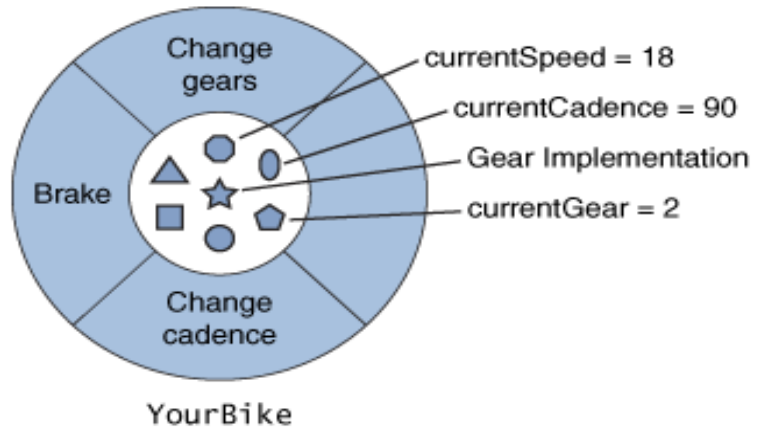
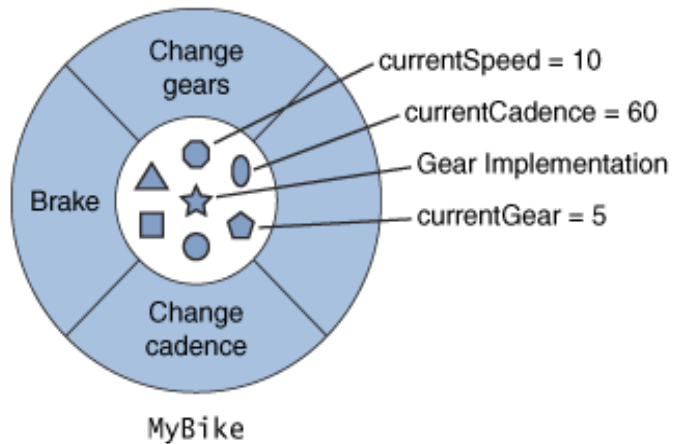
Exemplos

- A classe Bicycle
 - Quais itens descrevem características de uma bicicleta?
 - Quais itens descrevem comportamentos de uma bicicleta?



Exemplos

■ Objetos da Classe Bicycle



Relacionamento entre Classes

- Herança

- Classes “filhas” herdam o comportamento e atributos da classe “pai”.

- Composição

- Formação do todo pelas partes.

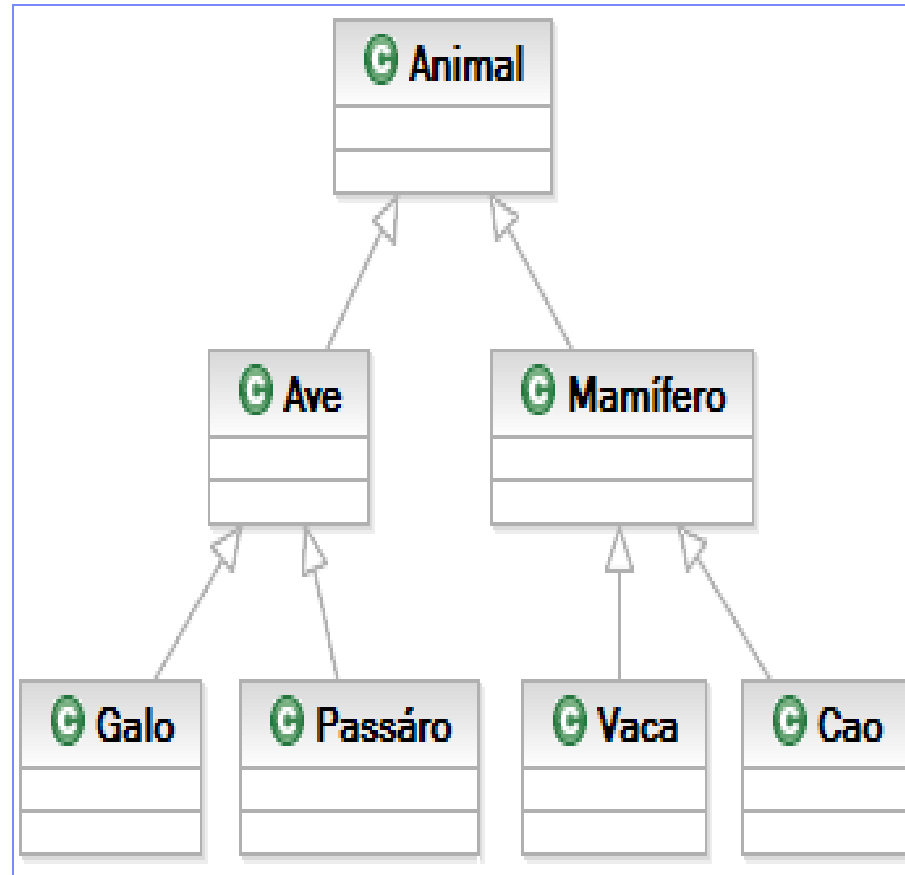
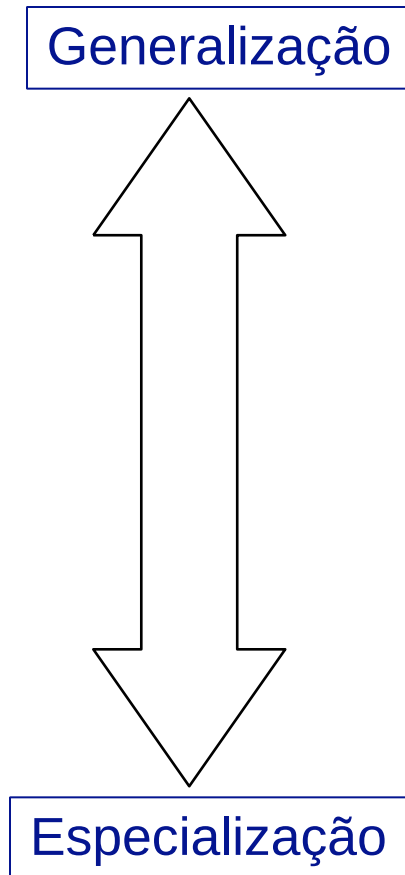
- Generalização

- Comportamento e características generalizados.

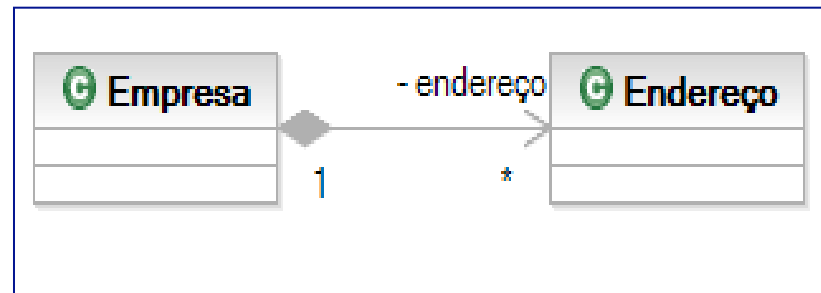
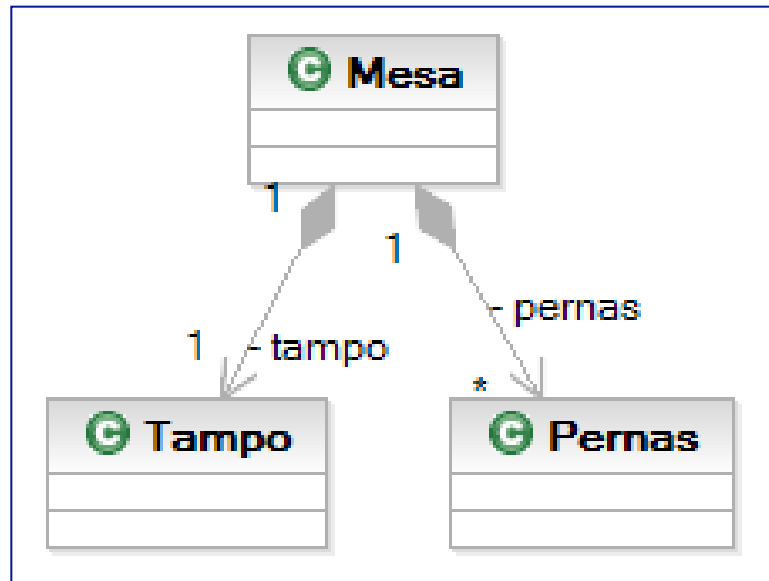
- Especialização

- Particularização do Comportamento das subclasses.

Herança, Generalização, Especialização



Composição



Classes Abstratas (abstract)

- Uma classe deve ser declarada abstrata, se ela não está completamente implementada:
 - ela contem métodos abstratos.
- Uma classe abstrata não pode ser instanciada:
 - pode ser estendida por uma subclasse que deve implementar todos os seus métodos abstratos (ou se tornar abstrata também).
- Classes abstratas são utilizadas para reunir classes em torno de métodos comuns.
 - P. ex., métodos matemáticos!

Interfaces

- Semelhantes a classes abstratas:
 - a interface não pode ter métodos concretos e só pode ter atributos constantes.
- Podem ser usada para simular herança múltipla.
- Representam idéias:
 - não é considerada uma classe.
 - as classes que implementam uma interface dão suporte a essa idéia.
- Os métodos abstratos de uma interface podem ser utilizados se são definidos em uma classe que implementa a interface.

Método construtor

- Utilizado para criar instâncias (objetos) de uma classe.

```
Lampada lp = new Lampada ();
```

- Responsável pela iniciação dos atributos.

```
Button botao = new Button  
("cachorro");
```

- Java executa os construtores do topo da hierarquia de classes até a classe atual.
- Se você não codifica um construtor, a linguagem provê um construtor padrão.

Threads

- Threads são processos leves implementados com um contador de programa, uma pilha independente e uma área de dados privados.
 - Embora só exista uma máquina virtual Java e na maioria dos casos, 1 só CPU, impossibilitando paralelismo real, o efeito do uso de threads é simular a existência de várias JVM, dando a ilusão de executarem ao mesmo tempo.
- Um processo real é em geral caracterizado como um programa em execução. Esse processo pode lançar várias threads que se responsabilizam pela execução de procedimentos específicos do programa.

Threads

- Um exemplo de thread da JVM é o coletor de lixo, o qual se responsabiliza pela liberação da memória dos objetos que não mais são usados.
- Toda thread em Java é uma instância da classe Thread. Elas podem estar em vários estados:
 - executando (de posse da CPU)
 - dormindo (por um tempo determinado),
 - bloqueado (em uma fila) esperando algum recurso,
 - pronto para executar (na fila do escalonador)
 - suspenso.

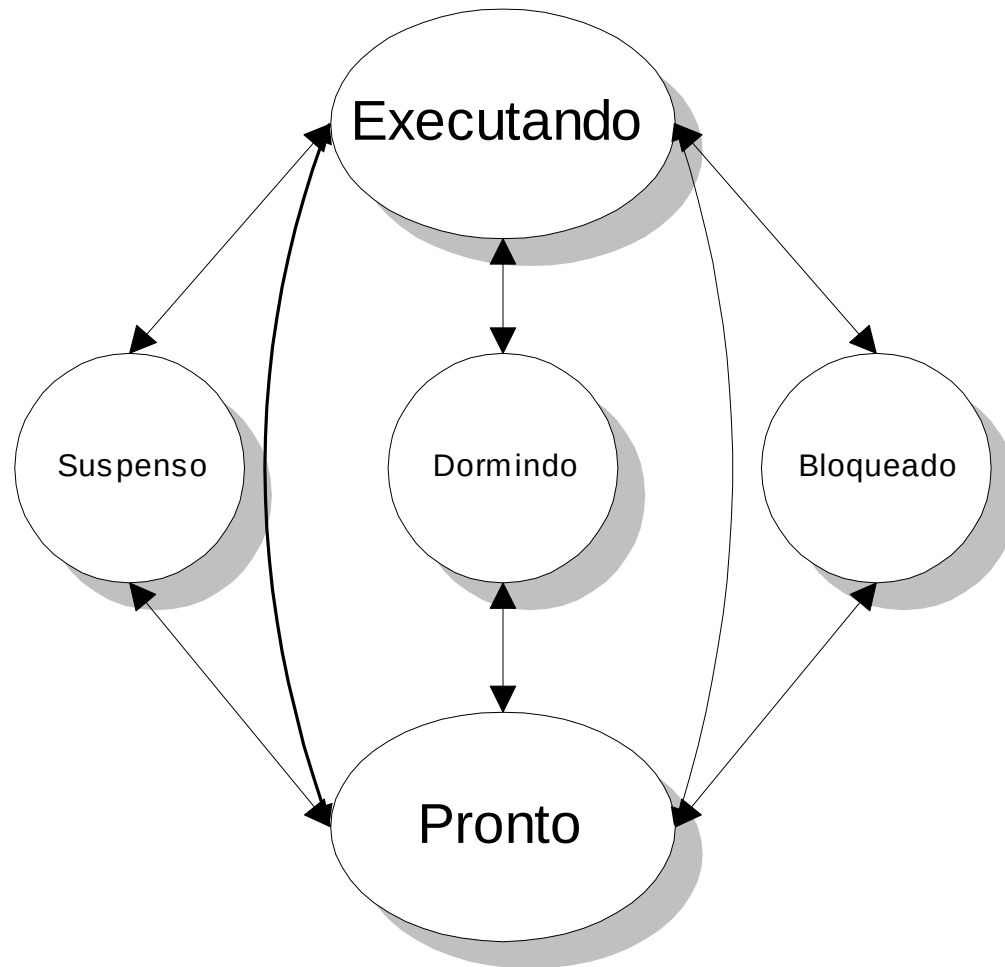
Threads

- Para executar uma thread, você deve chamar o método `start()`. Quando isso acontece, a thread não começa a sua execução imediatamente: ela é colocada na fila do escalonador de threads.
- Esse escalonador, utiliza uma política baseada em diversos fatores (p. ex.: prioridade, fato da thread ser ou não preemptiva, time slice, etc) para determinar a qual thread será alocada a CPU.

Controlando os Estados de uma Thread

- `yield()` : método estático definido em `java.lang.Thread` que coloca uma thread que esteja executando de volta ao estado "pronto para execução".
- `sleep()` : método estático que faz a thread passar uma quantidade de tempo sem fazer nada e sem consumir recursos da CPU. Lança uma exceção do tipo `InterruptedException` que deve ser tratada.
- bloqueio : quando um thread deve ficar esperando por um evento de E/S, a JVM bloqueia o método, para que outras tarefas possam ser realizadas.

Estados de uma Thread



Threads

Paralelismo e Sincronização

- Se duas threads se comunicam através de memória comum, o acesso deve ser controlado para permitir que uma única thread a acesse de cada vez.
 - Essa área de memória (denominada região crítica) pode controlar o acesso a um recurso compartilhado pelas threads.
 - se mais de uma thread tentar ter acesso à região crítica ao mesmo tempo o recurso poderá ser alocada a todas elas, gerando problemas!
 - Para evitar erros decorrentes dessa manipulação, declaramos uma parte do código (tipicamente um método, mas também é possível declarar blocos de código livres separados por chaves `{}`) como `synchronized`, e a partir de então somente um thread por vez poderá ter acesso aquele método. Essa forma de controle é conhecida por `lock`.

Threads

Paralelismo e Sincronização

- Exclusão mútua à região crítica pode ser feita com o uso de semáforos, wait e signal, monitor, etc.
 - Em Java, toda instância de objeto e toda classe tem um monitor associado.
 - Se você nunca usar os recursos de sincronização disponíveis, esse monitor nunca será alocado e estará eternamente a espera.
 - Um monitor é simplesmente um recurso de lock que serializa o acesso ao objeto ou a classe.
 - Para ganhar acesso, uma thread primeiro precisa ganhar controle do monitor.
 - Isso acontece toda vez que entramos em um método sincronizado (isto é, toda vez que um método declarado com a palavra-chave `synchronized` é executado por uma thread).
 - Uma thread esperando pela liberação de um monitor fica bloqueado.

Threads

Paralelismo e Sincronização

- Para liberar o acesso à região crítica, uma thread pode chamar a função `wait()`.
 - `wait()` só pode ser chamado se uma thread conseguiu o controle do monitor:
 - a chamada de `wait()` é feita dentro de um método sincronizado:
 - isso possibilita a sincronização de threads paralelas e a troca de informação entre elas.
- Exemplos de aplicações paralelas
 - operações com matrizes
 - resolução de expressões aritméticas,
 - problema do leitor vs escritor, etc.

Vantagens da Orientação a Objetos

- Reuso

- Acontece devido aos possíveis relacionamentos entre as classes: Herança e Composição

- Abstração

- Atributos e comportamentos bem encapsulados, o que torna o código mais manutenível e robusto



| Competência Java

A Linguagem Java

| Programando em Java

| Iniciando em Java

© 2006 IBM Corporation

Os Arquivos Fontes

- Arquivos fontes devem obrigatoriamente seguir a seguinte estrutura e ordem:
 - Definição do Pacote
 - Lista de import's
 - Declaração de Classe (s)
- É permitido que haja uma única classe pública por arquivo fonte
- A classe pública deve conter o mesmo nome que o arquivo fonte.
 - Se o nome da classe é Bicicleta então o nome do arquivo fonte deve ser Bicicleta.java

Arquivo Fonte

```
// Declaração de Pacote
package pessoal.meuPacote;

// Declaração de import's
import java.util.Random; //Importação de uma classe
import java.sql.*;        //Importação de um pacote inteiro

// Definições de Classes
public class MinhaClasse
{
    ...
}
```

Pacote

- Forma de organizar grupos de classes em unidades.
- Pode conter qualquer número de classes que se relacionam, seja pelo mesmo objetivo ou por escopo.
- Reduz problemas de conflitos de nome.
 - O nome de uma classe não é apenas aquele usado em sua declaração, mas sim o conjunto: nome do pacote + nome usado na definição da classe.
- Permite a proteção de classes, variáveis e métodos através dos modificadores.

Import

- Utiliza-se o import para declaração de classes que são referenciadas no arquivo fonte mas que não pertencem ao pacote onde este arquivo se encontra.
- Import's podem referenciar:
 - Outras classes no mesmo projeto
 - Classes da API Java, como java.util.List
 - Classes contidas nas bibliotecas utilizadas pelo projeto, ou seja, nos arquivos *.jar referenciados no classpath do projeto
- Sintaxe:
 - `import java.util.Date;`
 - `import java.util.*;`

Declarando uma Classe

- Definição de uma classe:

```
[modificadores] class NomeDaClasse  
{  
    ....  
}
```

- Exemplos:

```
public class Curso  
{  
}
```

```
public class Turma  
{  
}
```

```
public class Aluno  
{  
}
```

Declarando Atributos de Classe

- Definição de um Atributo:

```
[modificadores] tipo nomeDoAtributo [ = iniciação ];
```

- Exemplos:

```
private static int numero;
```

```
public final int tamanhoMaximo = 15;
```

```
private String nome = "Maria da Silva";
```

```
double raio = 6.5;
```

```
Object o = new Object();
```

Declarando Métodos

- Definição de um método:

```
[modificadores] retorno nomeDoMetodo ( [Argumentos] ) [ throws Exceções ]  
{  
    ...  
    [ return varRetorno; ]  
}
```

- Exemplos:

```
private void obtenNumeroAlunosTurma ( long codigoTurma ) { ... }
```

```
public String getNomeAluno( int codigoAluno ) { ... }
```

```
public void insereAluno ( String nomeAluno ) throws Exception { ... }
```

```
public static long getNumeroInstancias () { ... }
```

Exemplo de Classe

```
public class Aluno
{
    String nomeAluno;
    int codigoAluno;

    public String getNomeAluno()
    {
        return nomeAluno;
    }

    public void setNomeAluno( String param )
    {
        nomeAluno = param;
    }

    public int getCodigoAluno()
    {
        return codigoAluno;
    }

    public void setCodigoAluno( int param )
    {
        codigoAluno = param;
    }
}
```

Construtor da Classe

- Para que um objeto exista é necessário construí-lo, isto é, dizer para a JVM que é necessário espaço de memória para criação do objeto.
- Para contruir um objeto usa-se o construtor da classe.
 - `Aluno o1 = new Aluno();`
 - `Object o2 = new Object();`
- Toda classe possui, por default, um construtor padrão: público e sem argumentos.
- O construtor default somente é criado quando nenhum outro construtor for definido pelo programador.
- Uma classe pode ter quantos contrutores desejar.
- Implicitamente, ou mesmo explicitamente, o construtor sempre chama o contrutor da sua super classe.

Declarando Contrutores da Classe

- Definição de um método:

```
[modificador] nomeDaClasse ( [Argumentos] ) [ throws Exceções ]  
{  
    ...  
}
```

- Exemplos:

```
public Turma () { ... }
```

```
public Turma ( long codigoTurma ) { ... }
```

```
public Curso( int codigoCurso ) throws Exception { ... }
```

```
public Curso ( String nomeCurso, int codigoCurso ) { ... }
```

Tipos Primitivos

Tipo	Conteúdo	Default	Tamanho (bits)	Mínimo	Máximo
<code>boolean</code>	Valor lógico	<code>false</code>	8	--	--
<code>char</code>	Caracter Unicode	<code>\u0000</code>	16	<code>\u0000</code>	<code>\uFFFF</code>
<code>byte</code>	Inteiro com sinal	0	8	-2^7	$2^7 - 1$
<code>short</code>	Inteiro com sinal	0	16	-2^{15}	$2^{15} - 1$
<code>int</code>	Inteiro com sinal	0	32	-2^{31}	$2^{31} - 1$
<code>long</code>	Inteiro com sinal	0	64	-2^{63}	$2^{63} - 1$
<code>float</code>	Ponto flutuante	0.0	32	IEEE 754*	IEEE 754*
<code>double</code>	Ponto flutuante	0.0	64	IEEE 754*	IEEE 754*

Conversão de Tipos Primitivos

- Conversão: acontece quando existe uma modificação do tipo de uma variável de forma implícita.

- Atribuições

```
int i;  
double d;  
i = 100;  
d = i;
```

- Chamadas de Métodos

```
java.util.Vector v;  
...  
String s = "Eita!!"  
v.add(s);    // Definição do método add é add(Object o)  
...
```

- Operações Aritméticas

```
byte b = 2;  
int i = 5;  
float f = 11.1f;  
double d = b * i - f;    // O resultado final desta operação é um float  
                        // O valor é implicitamente convertido p/ double
```

Casting de Tipos Primitivos

- Casting: acontece quando há uma modificação do tipo de um variável de forma explícita.

- Exemplo 1:

```
short b1 = 2;  
short b2 = 5;  
short d = (short) b1 + b2;
```

...

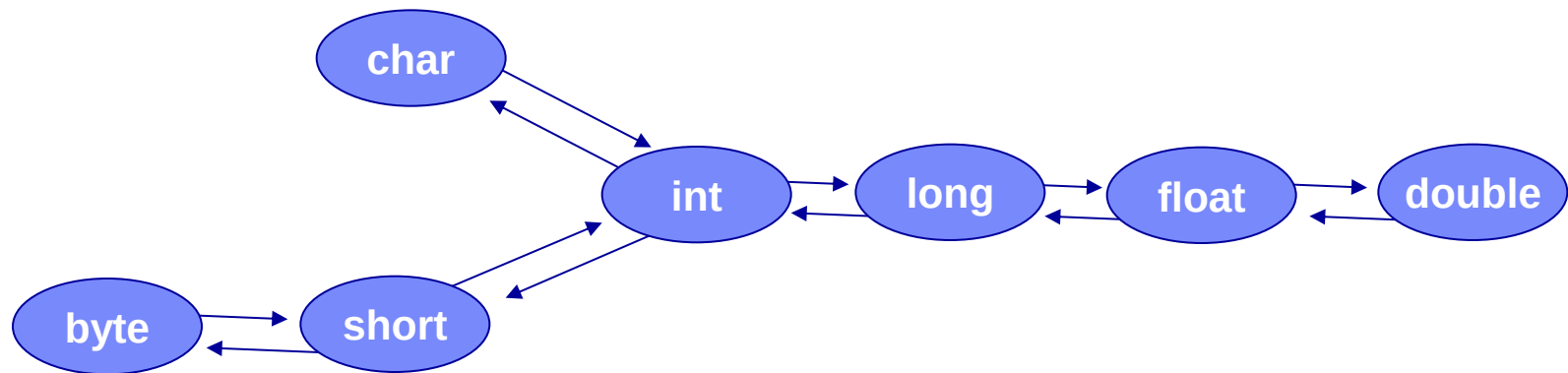
- Exemplo 2:

```
int i = 16777473;  
byte b2 = 5;  
byte b = (byte) i;
```

- Exemplo 3:

```
long l = 1200;  
int i = (int) l;
```

Conversão e Casting de Tipo Primitivos



Expressões e Literais

- Um **literal inteiro** é por default do tipo primitivo **int**.
- Um **literal ponto flutuante** é por default do tipo primitivo **double**.
- Durante a avaliação de expressões os operandos são convertidos para o mesmo tipo primitivo do operando cujo tipo possui maior precisão.
 - Tipos primitivos `short` e `byte` são convertidos para `int`.

```
byte b1 = 10;
```

```
byte b2 = 20;
```

```
short c = b1 + b2; → Erro de Compilação!
```

- Para corrigir é necessário um cast

```
short c = (short) b1 + b2;
```

Classes Wrappers

- São classes que encapsulam um único e imutável valor.
- Cada tipo primitivo possui uma classe wrapper correspondente.
- Permite armazenar os valores primitivos em estruturas da API Collection.

Tipo Primitivo	Classe Wrapper
boolean	Boolean
byte	Byte
char	Char
short	Short
int	Integer
long	Long
float	Float
double	Double

Usando as Classes Wrappers

- As classes wrapper podem ser criadas usando:
 - O valor primitivo a ser encapsulado:

```
Integer intObj1 = new Integer(1);  
Double doubleObj1 = new Double(5.5);
```
 - Uma representação em String do valor a ser encapsulado:

```
Integer intObj2 = new Integer("1");  
Double doubleObj1 = new Double("5.5");
```

A classe String

- String é uma classe da linguagem java e não um tipo primitivo.

- Exemplo:

```
String nome = "Carolina";
```

```
String sobreNome = "Buarque";
```

- Alguns métodos utilitários da classe String:

- `boolean equals(...)`

- `equalsIgnoreCase(...)`

- `int length(...)`

- `substring(...)`

- `toLowerCase(...)`

- `toUpperCase(...)`

- Referência para a classe String:

- API Java: <http://java.sun.com/j2se/1.4.2/docs/api/>

Modificadores de Acesso

- Definem o acesso que outras classes terão à classe, a seus métodos e seus atributos.
 - **public:** visível para qualquer classe sem restrições
 - Pode ser usado por classes, atributos e métodos
 - **protected:** visível para todas as classes do mesmo pacote e para as subclasses
 - Pode ser usado por atributos e métodos apenas
 - **Nenhum modificador definido (default):** visível para todas as classes do mesmo pacote
 - Pode ser usado por classes, atributos e métodos
 - **private:** visível somente para a classe
 - Pode ser usado por classes internas, atributos e métodos

A classe `Object`

- Topo da Hierarquia de classes do Java
 - Toda classe em Java é “filha” da classe `Object`.
- Mesmo que um classe não use a palavra reservada `extends`, o compilador gera a classe estendendo diretamente de `Object`.

A classe **Object**

- Métodos da classe **Object** que são herdados por toda classe:
 - Utilizados para controle de Threads.
 - `wait()`, `notify()`, `notifyAll()`
 - Utilizado para fornecer informações úteis sobre um objeto
 - `toString()`
 - Utilizado para realizar uma comparação mais detalhada entre dois objetos
 - `equals()`

A variável **this**

- **this** é uma referência para a instância corrente
- Utilizado em três situações:
 - Para diferenciar um atributo (variável da classe) de uma variável local ou de um parâmetro.
 - Para passar o objeto corrente como parâmetro
 - Para encadear chamadas de construtores.

A variável `this` – Diferenciando Atributos de Variáveis Locais

```
public class Pessoa
{
    private String nome;

    public void setNome( String nome )
    {
        // this.nome -> atributo da classe
        // nome -> variável local ao método
        this.nome = nome;
    }
}
```



| Competência Java

Sintaxe

| Programando em Java

| Iniciando em Java

© 2006 IBM Corporation

Sintaxe

- **Statement:**
 - Uma ou mais linhas terminadas por ‘;’
- **Blocos:**
 - Conjuntos de statements delimitados por ‘{’ e ‘}’
- **Comentários:**
 - `//` → comentário simples, de uma linha
 - `/* */` → comentário simples, de uma ou mais linhas
 - `/** */` → comentários para documentação (Javadoc)

Javadoc

- Ferramenta do Java para geração de documentação de código.
 - Como exemplo de javadoc tem-se a própria API da linguagem java em HTML com conteúdo originado dos arquivos fontes.
 - <http://java.sun.com/j2se/1.4.2/docs/api/>

- Exemplo de comentário javadoc:

```
/**  
 * @return String - Nome do usuário  
 *  
 */  
public String getNome() {  
    return this.nome;  
}
```

- Referência:
 - <http://java.sun.com/j2se/javadoc/>

Identificadores

- São os nomes dados a uma classe, método, atributo, variável ou parâmetro.
- Começam sempre por um caracter Unicode, (_) ou (\$).
- Diferenciam maiúsculas e minúsculas
- Não podem coincidir com uma palavra reservada.
- Identificadores Válidos - exemplos:
 - x
 - y
 - America
 - _9_i\$to_EH_meio_esquisito
 - \$4outroExemplo
 - exemploCOMmuitasPALAVRAS

Identificadores - Convenções da Linguagem

- Na linguagem Java é utilizada a seguinte convenção para formação de identificadores:
 - Constantes com todas as letras em maiúsculo: **CONSTANTE**;
 - `public static final int QUANTIDADE_MAXIMA = 100;`
 - Variáveis começam com letra minúscula: **variável**;
 - `String nomeUsuario;`
 - Classes começam com letra maiúscula: **Classe**;
 - `public class Usuario { ... }`
 - Métodos começam com letra minúscula: **metodo()**, **metodo2(int a)**;
 - `public void recuperaUsuario( int codigoUsuario ) { ... }`
 - Se nome for composto, cada nome começa com letra maiúscula: **variavelComNomeComposto**.
 - `String nomeUsuario;`

Palavras Reservadas

<code>abstract</code>	<code>boolean</code>	<code>break</code>	<code>byte</code>	<code>case</code>
<code>catch</code>	<code>char</code>	<code>class</code>	<code>const</code>	<code>continue</code>
<code>default</code>	<code>do</code>	<code>double</code>	<code>else</code>	<code>extends</code>
<code>false</code>	<code>final</code>	<code>finally</code>	<code>float</code>	<code>for</code>
<code>goto</code>	<code>if</code>	<code>implements</code>	<code>import</code>	<code>instanceof</code>
<code>int</code>	<code>interface</code>	<code>long</code>	<code>native</code>	<code>new</code>
<code>null</code>	<code>package</code>	<code>private</code>	<code>protected</code>	<code>public</code>
<code>return</code>	<code>short</code>	<code>static</code>	<code>super</code>	<code>synchronized</code>
<code>this</code>	<code>throw</code>	<code>throws</code>	<code>transient</code>	<code>true</code>
<code>try</code>	<code>void</code>	<code>volatile</code>	<code>while</code>	

throws vs throw e finally

```
public void setIdade(int idade) throws IdadeException {  
    if (idade < 0) {  
        throw new IdadeException("Idade fora da faixa");  
    }  
  
    ....  
}  
  
try {    //... código normal  
    return;  
} catch (Exception e) {  
    //... código que trata o erro }  
  
finally {  
    // código executado, mesmo levantando exceção } ;
```

Operadores

Prec	Operador	Operando	Assoc.	Operação
1	++ , --	A	D	In/decremento unário
	+ , -	A	D	Mais/menos unário (sinal)
	~	I	D	Complemento de 1
	!	B	D	Complemento lógico (not)
	(tipo)	O	D	"cast"
2	*, /, %	A,A	E	Multiplicação, divisão, modulo
3	+ , -	A,A	E	Adição, subtração
	+	S,S	E	Concatenação de strings
4	<<	I,I	E	Shift left
	>>	I,I	E	Shift right
	>>>	I,I	E	Shift right sem sinal
5	<, <=	A,A	E	Menor que, menor ou igual a
	> , >=	A,A	E	Maior que, maior ou igual a
	instanceof	O,C	E	Comparação de tipos

Operadores Continuação

Prec	Operador	Operando	Assoc.	Operação
6	<code>==, !=</code>	<code>P, P</code>	E	Igual/diferente (valores)
	<code>==, !=</code>	<code>O, O</code>	E	Igual/diferente (referência ao objeto)
7	<code>&</code>	<code>I, I</code>	E	E (bits)
	<code>&</code>	<code>B, B</code>	E	E (lógico)
8	<code>^</code>	<code>I, I</code>	E	XOR (bits)
	<code>^</code>	<code>B, B</code>	E	XOR (lógico)
9	<code> </code>	<code>T, T</code>	E	OU (bits)
	<code> </code>	<code>B, B</code>	E	OU (lógico)
10	<code>&&</code>	<code>B, B</code>	E	E (lógico)
11	<code> </code>	<code>B, B</code>	E	OU (lógico)
12	<code>?:</code>	<code>B, Q, Q]</code>	E	Operador condicional (ternário)
13	<code>=</code>	<code>V, Q</code>	D	Atribuição
	<code>*=, /=, %=, +=, -=, <<=, >>=, >>>=, &=, ^=, =</code>	<code>V, Q</code>	D	Atribuição com operação

Estruturas de Controle - Decisão

- A linguagem Java provê duas estruturas de decisão:

- `if() / else`

- `switch`

- `if() / else`

```
if ( expressao_boolean ) {  
    ....  
} [ else {  
    ...  
} ]
```

- `switch`

```
switch (key) {  
    case value:  
        <bloco de comandos>  
        break;  
    default :  
        <bloco de comandos>  
        break;  
}
```

Estruturas de Controle - Laço

- Existem três estruturas de controle em Java
 - `while()`
 - `do/while()`
 - `for`

Estruturas de Controle - `while()`

- Utilizado quando não se sabe de antemão a quantidade de iterações que serão executadas.
- Sintaxe:

```
while(expressao_booleana)
{
    <bloco de comandos>
}
```

- Caso a expressão booleana seja falsa, o bloco de código não será executado nenhuma vez.
- O programador deve garantir que a condição de parada será satisfeita em algum momento.

Estruturas de Controle - `do/while()`

- Utilizado quando não se sabe de antemão a quantidade de iterações que serão executadas.
- O trecho de código é sempre executado pelo menos uma vez.
- Sintaxe:

```
do
{
    <bloco de comandos>
} while(expressao_booleana)
```

- O programador deve garantir que a condição de parada será satisfeita em algum momento.

Estruturas de Controle - `for()`

- Utilizado quando sabemos de antemão o número de iterações que serão executadas.
- Sintaxe:

```
for (<statement_iniciacao> [, <statement_iniciacao n>];  
    <condicao_parada>;  
    <expressao_incremento> [, <expressao_incremento n>])  
{  
    <bloco de comandos>  
}
```

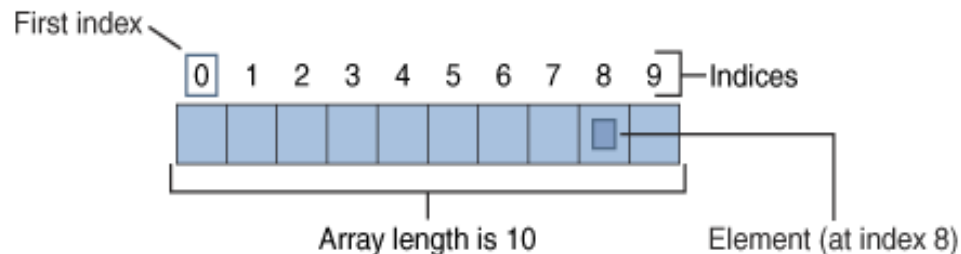
- O programador deve garantir que a condição de parada será satisfeita em algum momento.

Comandos **break**, **continue**

- **break:**
 - Comando de saída de um laço ou de um switch
- **continue:**
 - Comando de salto para a próxima iteração do laço.
- O comando **goto** não é implementado por Java.

Arrays

- Array é uma estrutura de tamanho fixo que armazena múltiplos valores do mesmo tipo.
- Qualquer tipo permitido em Java pode ser armazenado em um Array
 - Arrays de tipos primitivos
 - Arrays de referências de objetos
 - Arrays de outros arrays
- O tamanho de um Array precisa ser definido quando este é criado.
- Um elemento de um array correspondente a um dos elementos armazenados no array e pode ser acessado por sua posição.



Utilizando Arrays

- Para utilizar um array é necessário seguir os três passos abaixo:
 - Declaração
 - Construção
 - iniciação

Arrays - Declaração

- A declaração de uma array diz ao compilador o nome do array e o tipo de elemento que será armazenado.

```
int [ ] intArray;
```

```
String [ ] nomes;
```

```
Object [ ] objects;
```

- Nenhuma memória é alocada no momento da declaração do array.
- Não se pode estabelecer o tamanho do array no momento de sua declaração.

Arrays - Construção

- Construindo um array:

```
int [] intArray;
```

```
intArray = new int [10];
```

```
Object [] objArray = { "Objeto1", "Objeto2" };
```

```
String [][] stringMatrix = new String[10][20];
```

```
boolean[] answers = { true, false, true, true, false };
```

- Uma vez definido o tamanho do array este não pode mais ser alterado.
- Quando o array é de referências para objetos somente a memória ocupada pela referência em si é alocada. Nenhum objeto é criado neste momento.

Arrays - Construção

- Quando um array é contruído seus valores são automaticamente iniciados para valores padrão.
- Quando o array é de referências para objetos somente a memória ocupada pela referência em si é alocada. Nenhum objeto é criado neste momento.
- Valores default:

Tipo	Valor Inicial
<code>byte</code>	<code>0</code>
<code>short</code>	<code>0</code>
<code>int</code>	<code>0</code>
<code>long</code>	<code>0L</code>
<code>float</code>	<code>0.0f</code>
<code>double</code>	<code>0.0d</code>
<code>char</code>	<code>'\u0000'</code>
<code>boolean</code>	<code>false</code>
<code>reference</code>	<code>null</code>

Arrays - iniciação

- Declarando, contruindo e iniciando um array:

```
int[] anArray;           // declare an array of integers

anArray = new int[10];    // create an array of integers

// assign a value to each array element and print

for (int i = 0; i < anArray.length; i++) {

    anArray[i] = i;

    System.out.print(anArray[i] + " ");

}
```

- O menor índice do array é sempre zero.
- O maior índice do array é obtido através de `array.length - 1`

Modificadores

- **final**
 - Usado em:
 - Classes
 - Métodos
 - Variáveis
- **static**
 - Usado em:
 - Métodos
 - Variáveis
 - iniciadores estáticos
- **synchronized**

Modificador **final**

- Classe: define que uma classe não pode ser estendida.

```
public final class Math {  
    // esta classe não pode ser estendida  
}
```

- Método: define que um método não pode ser sobrescrito.

```
public final void metodoFinal() {  
    // este método não pode ser sobre-escrito  
}
```

- Variável: define que uma variável não pode ser modificada depois de receber um valor.
 - Para tipos primitivos não podem receber outro valor
 - Para referências, não podem referenciar um outro objeto, mas o conteúdo do objeto original pode ser alterado.

Modificador `static` – Variáveis

- Variáveis: define que uma variável terá somente uma instância em toda máquina virtual. Uma variável estática é uma variável de classe.
 - **Variável de Classe:** variável comum a todos os objetos da classe. Isto é, uma alteração no valor desta variável por um dos objetos é refletida em todos objetos, pois todos estão enxergando a mesma variável.
 - **Variável de Instância:** cada objeto possui um valor diferente setado na variável.

Modificador `static` – Métodos

- Métodos: define que o método se refere à classe e não a alguma instância de uma classe.

```
double angulo = 3.14;
```

```
double cosseno = Math.cos( angulo );
```

Modificador `static` – iniciador Estático

- iniciadores estáticos: trecho de código que é executado uma única vez. Quando a classe é carregada.

```
public class iniciadorEstatico {  
    private static String staticString = null;  
    private String string = null;  
    static {  
        staticString = "classe foi carregada";  
        string = "esta linha não compila";  
    }  
}
```

Modificador `synchronized`

- Utilizado para controlar o acesso em trechos críticos de código, para programas multi-threaded.
- Define que apenas uma thread poderá executar o trecho delimitado pelo `synchronized` num período de tempo.

```
public synchronized void metodoSincronizado() {  
    /* somente uma thread de cada vez pode executar este  
    trecho */  
}
```

Todos os Modificadores

Modificador	Classe	Atributo	Método	Construtor	Blocos livres
<code>public</code>	sim	sim	sim	sim	não
<code>protected</code>	não	sim	sim	sim	não
(default)	sim	sim	sim	sim	sim
<code>private</code>	não	sim	sim	sim	não
<code>final</code>	sim	sim	sim	não	não
<code>abstract</code>	sim	não	sim	não	não
<code>static</code>	não	sim	sim	não	sim
<code>native</code>	não	não	sim	não	não
<code>transient</code>	não	sim	não	não	não
<code>volatile</code>	não	sim	não	não	não
<code>synchronized</code>	não	não	sim	não	sim



| Competência Java

Refêrencias e Objetos - Memória

| Programando em Java

| Iniciando em Java

© 2006 IBM Corporation

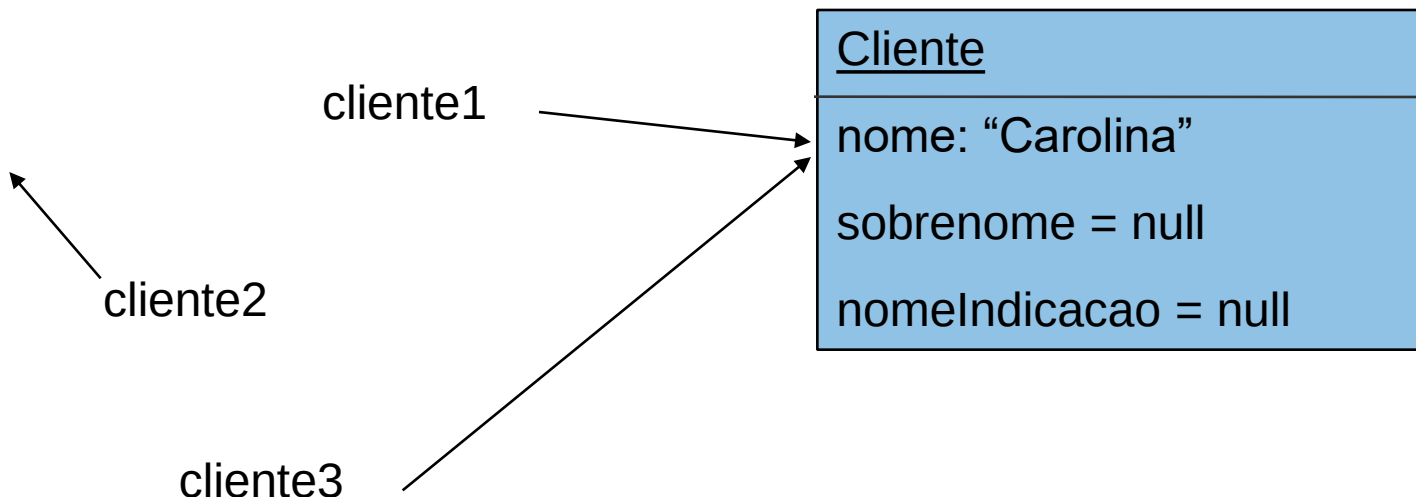
Modelo de Memória

- O modelo de memória do Java é baseado na abordagem de dupla indireção, isto é, as referências são endereços de outro endereço.
- Esta abordagem permite a utilização do garbage collector para realocação de memória e redução da fragmentação.

Referências de Objetos - Atribuições

- Atribuições entre variáveis de um mesmo tipo não criam novos objetos, mas sim cópias da referência para o **mesmo objeto**.

```
Cliente cliente1 = new Cliente();  
cliente1.setNome("Carolina");  
Cliente cliente2 = null;  
Cliente cliente3 = cliente1;
```



Igualdade entre Objetos

- A comparação em Java entre dois tipos primitivos se faz a partir do operador `==`.
- A utilização do operador `==` para comparar objetos pode gerar desigualdades para comparar objetos idênticos.
- No caso das `String`'s para resolver este problema basta utilizar o método `equals`.
- Todas as classes de Java estendem a classe `Object` que possui o método `equals` com a seguinte assinatura:
 - `public boolean equals( Object obj )`
- Para comparar dois objetos de uma mesma classe, deve-se sobrescrever o método `equals` de maneira que ele possua a funcionalidade de comparar se o objeto recebido é igual ao atual.

Referências e Objetos - Passagem de Parâmetros

- Quando um argumento é passado como parâmetro na chamada de uma função, na realidade um cópia do argumento é passada.
 - **Tipos Primitivos:** é passado uma cópia do tipo primitivo. Caso este valor seja alterado dentro do método, isso não afetará o valor no método original.
 - **Referências a Objetos:** é passado uma cópia da referência.
 - Caso a referência seja alterada para outro objeto, isso não afetará a referência original.
 - Caso o objeto apontado pela referência seja alterado, essa alteração será perpetuada.
- No método não é possível modificar a referência, mas é possível modificar o objeto referenciado.

Passagem de Parâmetros - Exemplo

```
public class TesteParametro
{
    public static void modificaSB1( StringBuffer aString) {
        aString = new StringBuffer("nova string");
    }
    public static void modificaSB2( StringBuffer sb ) {
        sb.append("-realizado append");
    }
    public static void modificaInt1( int i ) {
        i+= 10;
    }
    public static void modificaInt2( int[] array ) {
        for(int i = 0; i < array.length; i++ ) {
            array[i]+=10;
        }
    }
}
```

Passagem de Parâmetros - Exemplo

```
public static void main( String[] args ) {  
    StringBuffer string = new StringBuffer("TESTE");  
    modificaSB1(string);  
    System.out.println("Apos execucao modificaSB1: " + string );  
    modificaSB2( string );  
    System.out.println("Apos execucao modificaSB2: " + string );  
    int[] arrayInt = { 10 };  
    modificaInt1( arrayInt[0] );  
    System.out.println("Apos execucao modificaInt1: " +  
        arrayInt[0] );  
    modificaInt2( arrayInt );  
    System.out.print("Apos execucao modificaInt2: ");  
    for( int j = 0; j < arrayInt.length; j++ ) {  
        System.out.println( arrayInt[j] + ", " );  
    }  
}
```

Passagem de Parâmetros - Resultados

Apos execucao modifySB1: TESTE

Apos execucao modifySB2: TESTE-realizado append

Apos execucao modifyInt1: 10

Apos execucao modifyInt2: 20,

Passagem por Valor - Exemplo

```
class Retangulo{
    //Atributos da Classe
    float orig_x, orig_y;
    float altura, largura;
    //Método da classe
    public void translacao (float x, float y) {
        //Realiza translação
        orig_x = x;
        orig_y = y;
        //Altera valores dos argumentos
        x = 0.0;
        y = 0.0;
    }

    public static void main( String args[] ){
        Retangulo ret = new Retangulo();
        float x1 = 15.5;
        float y1 = 10.5;
        ret.translacao(x1, y1);
        System.out.println("O valor de x1 é " + x1);
        System.out.println("O valor de y1 é " + y1);
    }
}
```

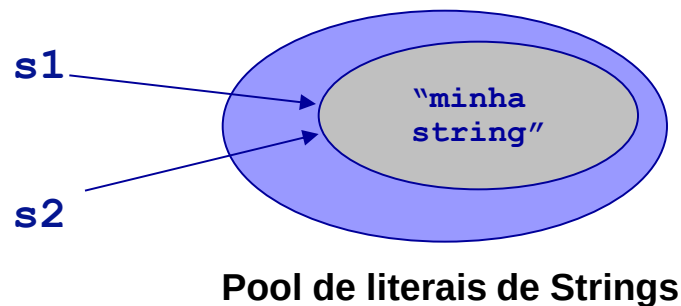
Passagem por Referência - Exemplo

```
class Vetor{
    //Variável global da classe
    String mensagem;
    //Método de iniciação do vetor
    public void inicia(){
        //Declaração e iniciação do vetor
        int v[] = {1, 2, 3, 4, 5};
        //Mensagem a ser escrita na tela
        mensagem = "Os valores originais do vetor são: ";
        for (int i = 0; i < v.length; i++){
            mensagem += " " + v[i];
        }
        /* Chamada ao método modificaVetor, passando v1 como argumento.
           Passagem por referência */
        modificaVetor( v );
        mensagem += "\nOs valores do vetor após modificação são: ";
        for (int i = 0; i < v.length; i++){
            mensagem += " " + v[i];
        }
        modificaElemento( v[2] );
        mensagem += "\n O valor de v[2] é =" + v[2];
    }
    //Modifica todos os elementos do vetor. Multiplica todos por 3
    public void modificaVetor( int v1[] ){
        for (int j = 0; j < v1.length; j++){
            v1[j] *= 3;
        }
    }
    //Modifica um elemento, multiplicando-o por 3
    public void modificaElemento( int elem ){
        elem *= 3;
    }
    //Retorna a mensagem
    public String getMensagem(){
        return mensagem;
    }

    public static void main( String args[] ){
        Vetor v = new Vetor();
        v.inicializa();
        System.out.println( v.getMensagem() );
    }
}
```

A classe String

- Java utiliza esta classe para encapsular strings de caracteres.
- Cada instância da classe representa uma string imutável, ou seja, depois de criada, a string representada não pode ser alterada.
- Quando uma literal é compilada, ela é adicionada a um pool de literais. Caso o compilador encontre essa literal no pool, a **literal existente é re-utilizada**.



Garbage Collection

- Em Java, não é preciso fazer alocação dinâmica explícita para criar objetos, também não é preciso desalocar memória.
- O Garbage Collector é um processo interno da JVM que de tempos em tempos executa seu processo e faz desalocação de objetos que não possuem mais referências.
 - Não é possível saber quando os objetos serão enviados para o Garbage Collector.
 - Algumas classes precisam definir como suas instâncias devem ser excluídas da memória.
 - Para isto, o GC utiliza o método `finalize()` da classe `Object`. É possível estender o método para fazer a desalocação personalizada para uma subclasse.



| Competência Java

Herança e Java

| Programando em Java

| Iniciando em Java

© 2006 IBM Corporation

Orientação a Objetos - Vantagens

- Reuso

- Herança
- Composição

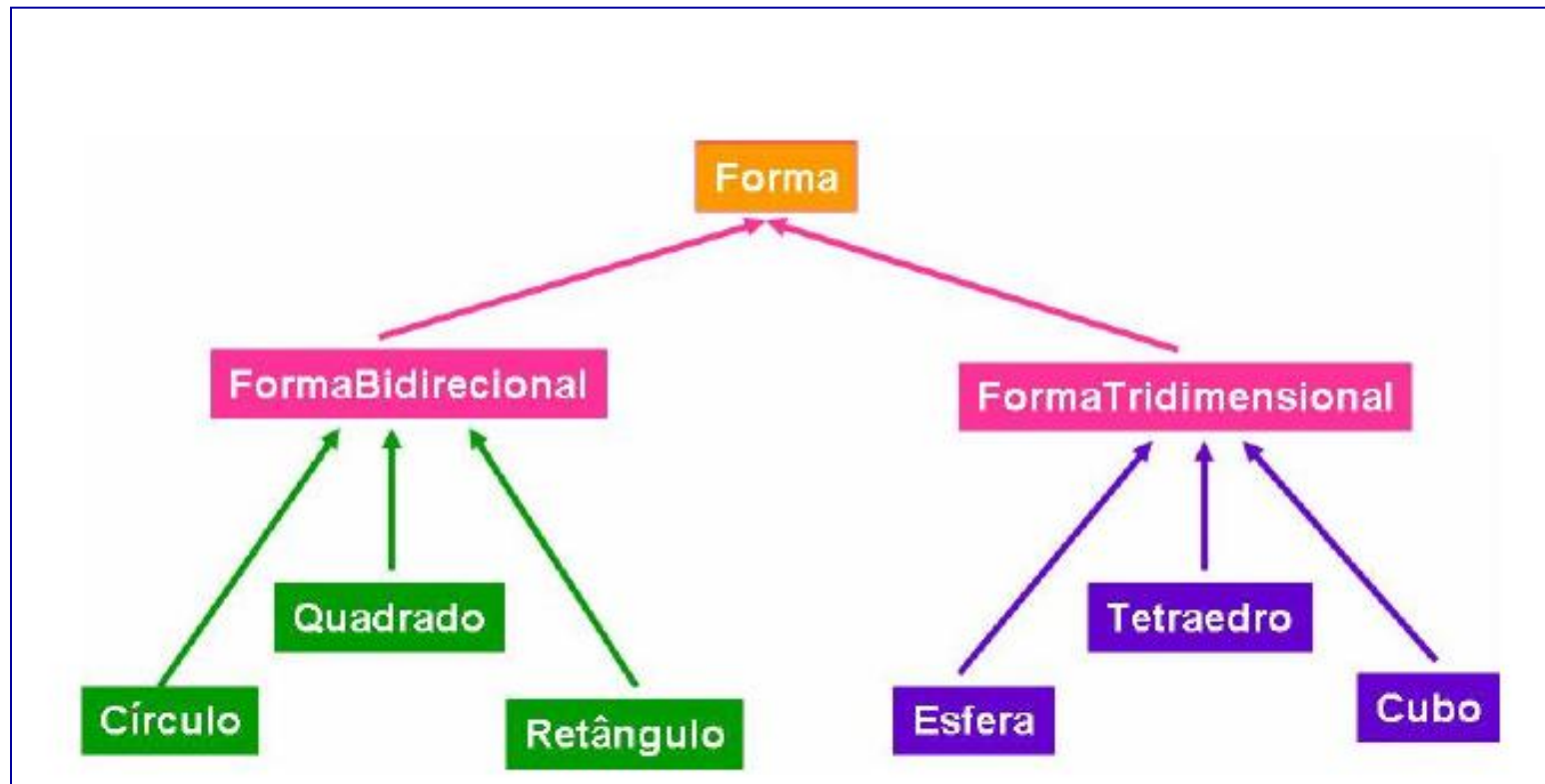
- Abstração

- Interfaces
- Classes Abstratas
- Encapsulamento

Herança

- A implementação do conceito de herança em Java é feito com a palavra **extends**.
- Em Java é possível estender SOMENTE uma classe.
- Um classe que estende a outra é chamada de sub classe, e a classe estendida é chamada de super classe.
 - A sub classe é uma especialização da super classe.
 - A super classe é uma generalização da sub classe.

Herança

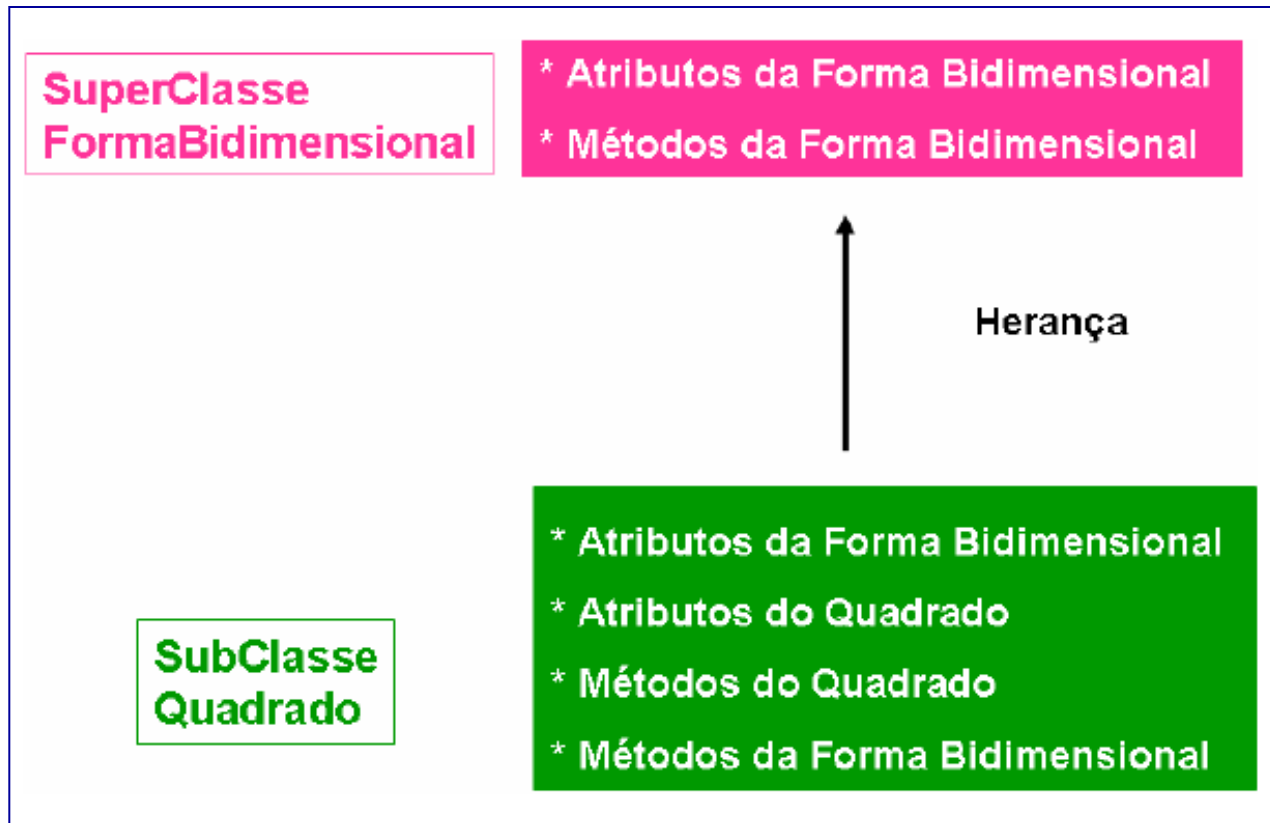


Herança - Sintaxe

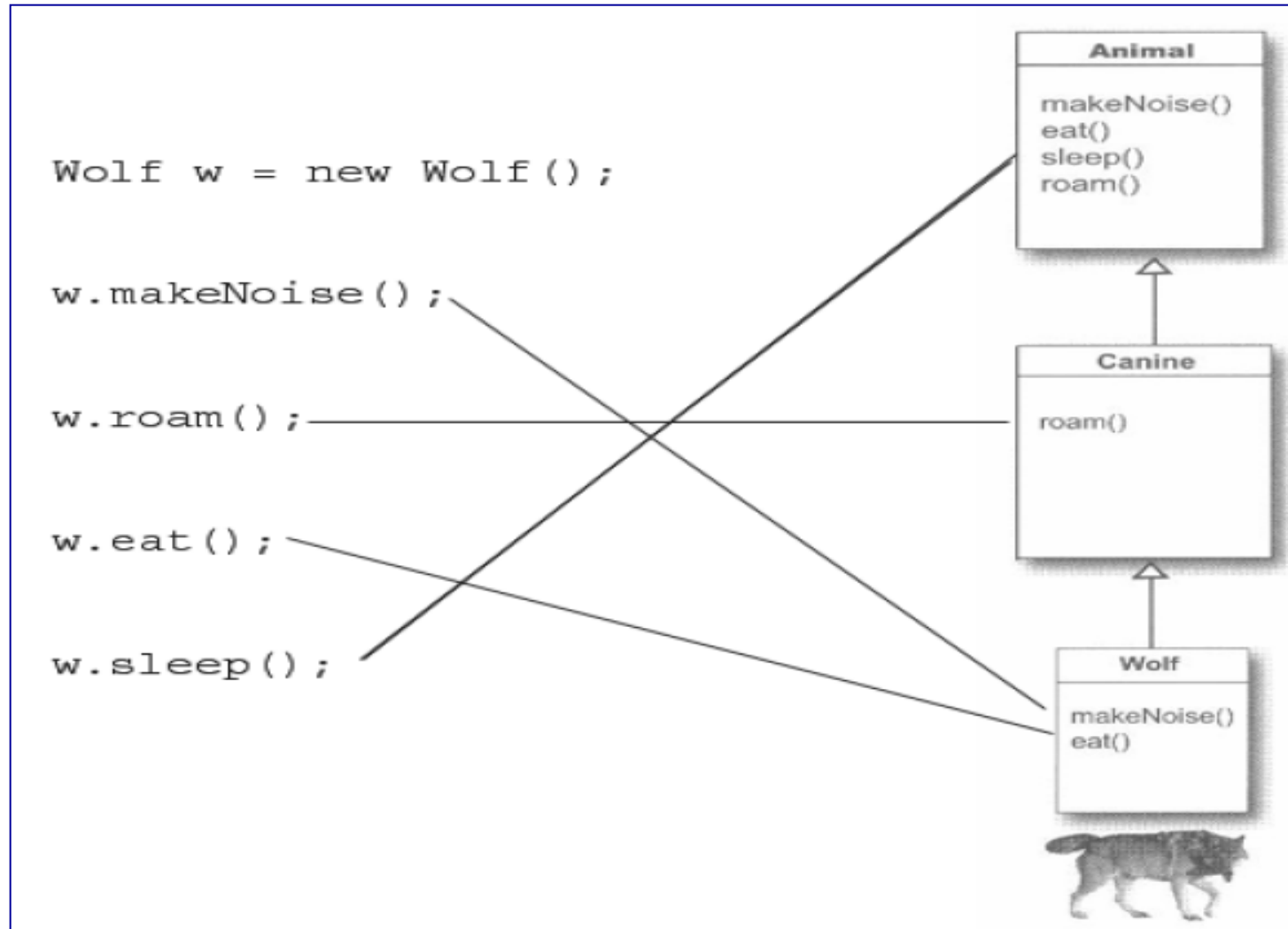
```
public class Pessoa
{
    public void getNome()
    {
    }
    public void getCPF()
    {
    }
}
```

```
public class Funcionario
    extends Pessoa
{
    public void getSalario()
    {
    }
}
```

Herança



Herança - Sobreescrita de métodos



Sobreescrita de Métodos

- Usado quando é preciso modificar o comportamento de um método da classe pai.
- Para sobreescrever um método as condições abaixo devem ser satisfeitas:
 - O nome do método assim como o tipo e a ordem dos parâmetros devem ser idênticos aos do método da classe pai.
 - O tipo de retorno também deve ser idêntico.
 - A visibilidade não deve ser mais restritiva que a visibilidade do método original.
 - O método não deverá lançar “checked exceptions” que não são lançadas pelo método original.

Herança e **super**

- Quando uma classe sobreescreve um método da super classe, o comportamento normal da subclasse é executar o novo método.
- Existe uma maneira de executar o método da super classe através da palavra reservada **super**, que é uma referência à super classe.

```
public void metodoSobrescrito  {  
  
    super.metodoSobrescrito();  
  
    // outras coisas específicas da classe filha  
  
}
```

Herança - Construtores

- Construtores não são herdados como métodos comuns, e devem ser definidos para cada classe.
- Caso uma classe não tenha nenhum construtor definido, o compilador automaticamente cria um construtor default que simplesmente chama o construtor da superclasse.
- Também é possível chamar explicitamente o construtor da superclasse através do método `super()`.
 - Caso o método `super()` seja chamado no construtor da subclasse, esta chamada deve ser o primeiro comando do construtor.

Sobrecarga de Métodos

- **Usada quando é preciso vários métodos que desempenham papéis semelhantes em diferentes condições.**
- **Para sobrecarregar um método as seguintes condições devem ser satisfeitas:**
 - **A identidade de um método é determinada pelo nome completo da classe a que pertence, pelo seu nome, pelo seu tipo, ordem e quantidade dos parâmetros.**
 - **Dois ou mais métodos na mesma classe (incluindo métodos da super classe) com o mesmo nome mas com uma lista de parâmetros diferentes são métodos sobrecarregados.**
 - **O tipo de retorno do método, sua visibilidade, e lista de parâmetros pode variar livremente.**
 - **Métodos sobrecarregados podem chamar um ao outro, utilizando uma chamada comum de método com lista de parâmetros apropriada.**

Sobrecarga de Métodos

```
public class ExemploSobrecarga()  
{  
    public void metodoSobrecarregado(int param1){ }  
    public void metodoSobrecarregado(String param1){ }  
    public void metodoSobrecarregado(int param1,  
                                     String param2){ }  
    public void metodoSobrecarregado(String param1,  
                                     int param2){ }  
    public void metodoSobrecarregado(double param1,  
                                     String param2, int param3){ }  
}
```

Sobrecarga de Construtores

```
class Base {  
    public Base( String s ) {  
        // inicia o objeto usando s  
    }  
    public Base( int i ) {  
        // inicia o objeto usando i  
    }  
}  
  
class Derived extends Base {  
    public Derived( String s ) {  
        // passa o controle para o construtor de Base na linha 2  
        super( s );  
    }  
    public Derived( int i ) {  
        // passa o controle para o construtor de Base na linha 5  
        super( i );  
    }  
}
```

Orientação a Objetos - Abstração

- Um dos maiores benefícios do paradigma OO é a noção de um tipo de dado abstrato
 - Pense na classe *java.lang.String* e todos os métodos que ela possui e podem ser utilizados pelo programador.
 - É importante para o programador saber como foram implementados os métodos desta classe?
 - É importante saber se a *String* foi implementada utilizando um *array* ou uma lista ligada?
 - Para o programador, o importante é somente saber que os métodos existem

OO - Bom uso da Abstração

- A abstração é um elemento chave de um bom código orientado a objetos e deve ser utilizado em qualquer projeto
- O primeiro objetivo em definir uma boa classe não é pensar na sua implementação, mas sim nas suas responsabilidades e comportamento, ou seja, nos seus métodos públicos
- Todo o comportamento da classe deve ser acessado através de métodos.

OO - Bom uso da Abstração

- Somente depois de definidas as responsabilidades de uma classe é que a preocupação com a sua implementação deve ser levantada
- A implementação da classe não irá importar para quem a utilizará, portanto ela deve estar bem encapsulada
 - Atributos e variáveis internas todos privados

Num bom uso da abstração, a implementação de uma classe poderia ser escolhida na hora de execução

Abstração e Java

- Em Java, o conceito de abstração é implementado com:
 - Interfaces
 - Classes abstratas
 - Métodos abstratos

Interfaces

- Não são classes
- Não possuem métodos implementados
- Possuem apenas definição de comportamento:
 - Métodos abstratos
 - Constantes
- Não podem ser instanciadas

Para que servem???

Interfaces

- Mesmo não existindo implementação as interfaces definem um comportamento.

- Definição da interface Pet - definição de um comportamento:

```
public interface Pet
{
    public abstract void beFriendly();
    public abstract void play();
}
```

- Implementação da interface Pet:

```
public class Dog implements Pet {
    public void beFriendly(){ ... }
    public void play(){ ... }
}
```

Interfaces - Abstração

- Como é uma classe comum, pode ser instanciada:
- `Dog pet = new Dog();`
 - Não gera erro, porém não usa o conceito de abstração, força a utilização de `Dog`
- `Pet pet = new Dog();`
 - Usa o conceito de abstração, pois o código não depende da implementação do `Pet`.

Classes Abstratas

- Classes abstratas são classes que podem possuir métodos implementados, mas que possuem ao menos um método abstrato.
 - Método abstrato é aquele em que não existe implementação, apenas sua definição.
- São úteis quando definem a implementação de métodos comuns a todas as classes que as estendem, mas obrigam que cada uma destas classes defina a implementação dos outros métodos abstratos.
- Classes abstratas não são instanciadas.

Classes Abstratas

```
public abstract class Animal {  
    public void comer() { ... }  
    public abstract void andar();  
}  
  
public class Cachorro extends Animal {  
    public void andar() {  
        // anda com quatro patas  
    }  
}  
  
public class Canguru extends Animal {  
    public void andar() {  
        // anda com duas patas - pulando  
    }  
}
```

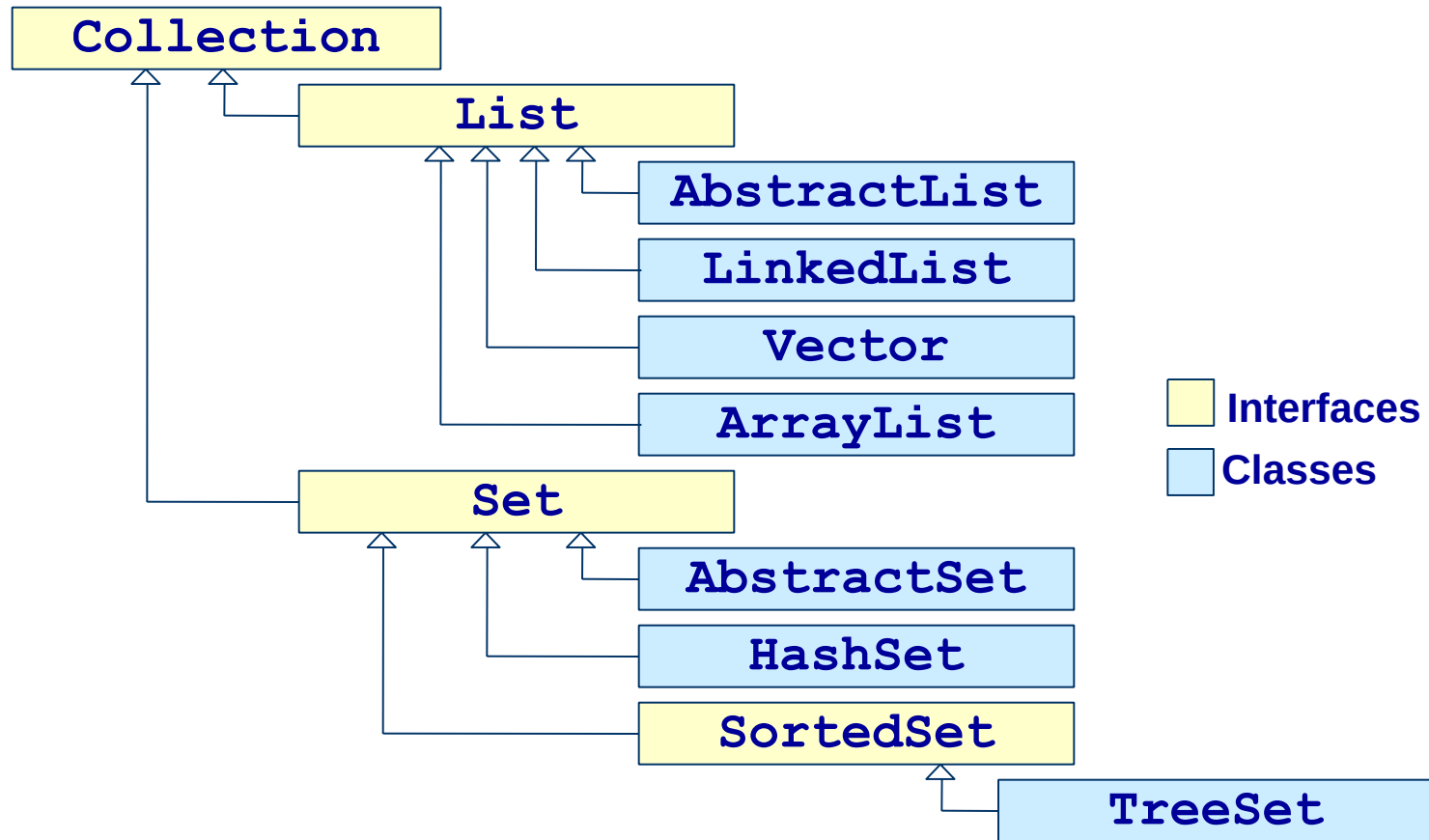
Classes Abstratas X Interfaces

- Interfaces possuem somente métodos abstratos
- Classes abstratas possuem métodos abstratos, mas também possuem tantos métodos implementados quantos for necessário.

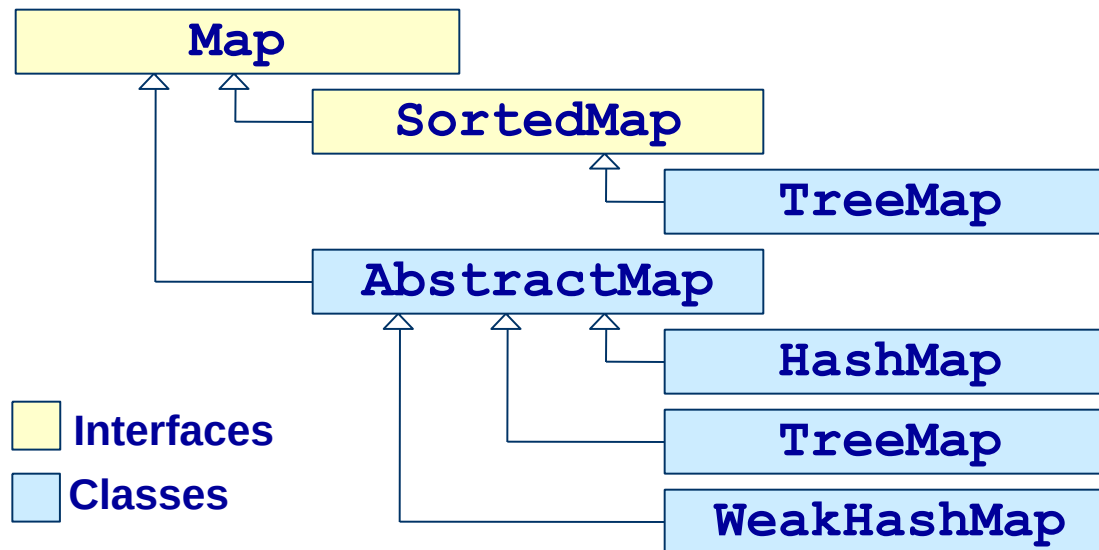
Interfaces – API Collection

- Um bom exemplo de interfaces que utilizamos sempre é a API Collection, um conjunto de classes do pacote *java.util*
- Possui basicamente 4 tipos de interfaces:
 - Collection – coleção genérica de objetos
 - List – lista de objetos
 - Set – conjunto de objetos (sem repetição)
 - Map – mapeia chave para valores (objetos)

Collection



Map



List e Map

■ List:

- `add(int index, Object element)`
- `add(Object o)`
- `get(int index)`
- `remove(int index)`

■ Map:

- `put(Object key, Object value)`
- `get(Object key)`

Percorrendo uma Collection

- Exemplo:

```
public void percorre() {  
    Collection c = new ArrayList();  
    Iterator it = c.iterator();  
    while ( it.hasNext() ) {  
        Object ob = it.next();  
        System.out.println( ob.toString() );  
    }  
}
```

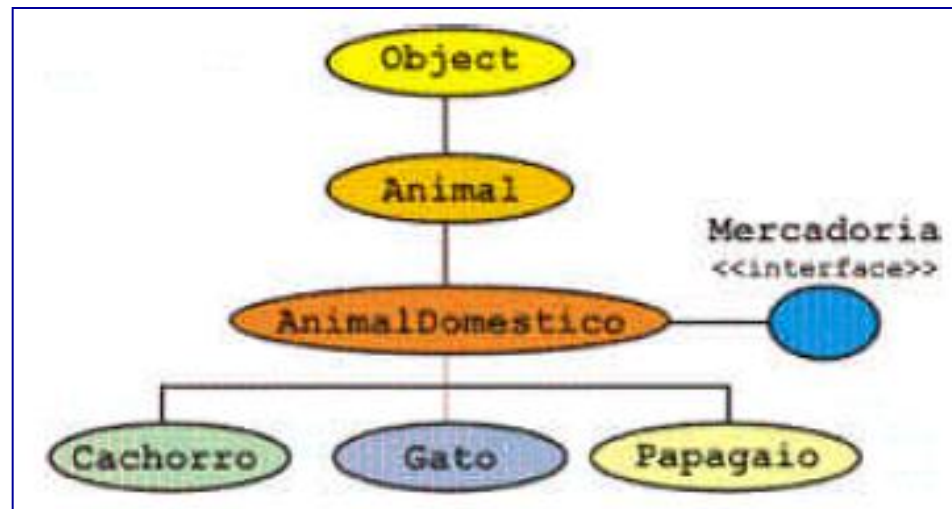
Percorrendo um List

- Exemplo:

```
public void percorre()  
{  
    List l = new ArrayList();  
    for ( int i = 0; i < l.size(); i++ )  
    {  
        Object ob = l.get(i);  
        System.out.println( ob.toString() );  
    }  
}
```

Conversão/Casting de Referências a Objetos

- **Conversão:** acontece quando há uma modificação do tipo de um variável de forma **implícita**.
- **Casting:** acontece quando há uma modificação do tipo de uma variável de forma **explícita**.
- Analise a figura abaixo:



Conversão de Referências de Objetos

■ Exemplos OK

- De acordo com a figura anterior todos os itens abaixo ilustram atribuições onde ocorre conversão implícita:

```
Gato g = new Gato();
```

```
AnimalDomestico ad = g;
```

```
Object o = g;
```

```
Papagaio p = new Papagaio();
```

```
Mercadoria m = p;
```

```
Object o = m;
```

Conversão de Referências de Objetos

■ Exemplos ERRO

- De acordo com a figura anterior todos os itens abaixo ilustram atribuições onde ocorre não conversão implícita:

```
Gato g = new Gato();
```

```
Cachorro c = g;
```

```
Object o = new Object();
```

```
Animal a = new Animal ();
```

```
Papagaio p = a;
```

Casting de Referências de Objetos

```
Animal a1 = new Animal;  
Papagaio p1 = new Papagaio();  
Papagaio p2 = new Papagaio();  
  
AnimalDomestico ad = new AnimalDomestico();  
Gato g1 = new Gato();  
Gato g2 = new Gato();  
a1 = p2;                // Conversão implícita  
p1 = (Papagaio) a1;      // Casting válido  
ad = g1;                 // Conversão válida  
g2 = (Gato)a1;           // Casting Legal (compilação)...  
                        // Erro em execução
```

Conversão de Referências de Objetos - Válidas

	ClasseTipoOriginal Classe	ClasseTipoOriginal Interface	ClasseTipoOriginal Array
ClasseNovoTipo Classe	ClasseTipoOriginal deve ser uma subclasse de ClasseNovoTipo	ClasseTipoOriginal Deve ser um Object	ClasseTipoOriginal deve ser um Object
ClasseNovoTipo Interface	ClasseTipoOriginal deve implementar ClasseNovoTipo	ClasseTipoOriginal deve ser uma subinterface ClasseNovoTipo	ClasseTipoOriginal deve ser Cloneable OU Serializable
ClasseNovoTipo Array	Erro	Erro	

Casting de Referências de Objetos - Válidas

	TipoOriginal Classe “não-final”	TipoOriginal Interface	TipoOriginal Interface	TipoOriginal Array
NovoTipo Classe “não-final”	TipoOriginal deve herdar de NovoTipo ou vice-versa	TipoOriginal deve herdar de NovoTipo	Sempre OK	TipoOriginal deve ser Object
NovoTipo Classe “final”	NovoTipo deve herdar de TipoOriginal	TipoOriginal e NovoTipo devem ser da mesma Classe	NovoTipo deve implementar uma interface ou Serializable	Erro de Compilação
NovoTipo Interface	Sempre OK		Sempre OK	Erro de Compilação
NovoTipo Array	NovoTipo deve ser um Object	Erro de Compilação	Erro de Compilação	TipoOriginal contém objetos que possam ser casting para os objetos de NovoTipo

Operador `instanceof`

- Para auxiliar a operação de casting, existe uma maneira para saber qual a real instância de uma referência, e testar para saber se vai ocorrer um erro numa suposta conversão explícita

```
Object s1 = new String("objeto do tipo String");  
if ( s1 instanceof String ) {  
    String s = (String) s1;  
    // s.doSomething();  
}
```



| Competência Java

Exceptions

| Programando em Java

| Iniciando em Java

© 2006 IBM Corporation

Exceptions

- Usadas para controle, durante a execução de um programa, de alguma coisa que não é normal - do ponto de vista do objetivo a ser alcançado - mas que possa acontecer.
- Tipos de erro que podem acontecer:
 - Previsíveis:
 - usuário entra com nome de arquivo inválido, falha num componente de rede, arquivo corrompido, etc.
 - Não previsíveis
 - *bugs* no programa, acesso a posições inexistentes num *array*, falta de memória do sistema, etc.

Exceptions

- Para os problemas previsíveis, o modo de tratamento destes erros em linguagens de programação mais tradicionais, devem ser feitas com “if”s.
- Exemplo: no caso do nome do arquivo, deve se testar se o arquivo existe antes de efetivamente usá-lo.
- Esse tipo de tratamento deixa o código extenso e complexo. O ideal seria fazer um tratamento devido somente se o problema venha a ocorrer.
- No caso de problemas que não são previstos, ainda assim seria interessante uma forma de tratamento especial para avisar o usuário sobre estes problemas.

Exceptions

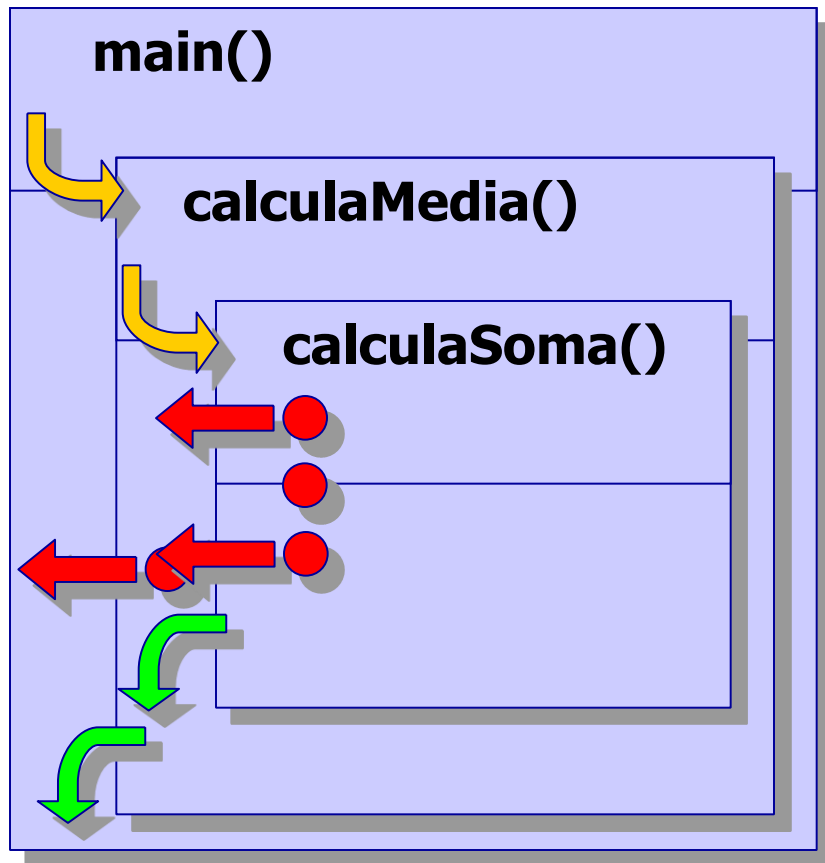
- Em Java, existe um mecanismo que faz exatamente isso. Este mecanismo funciona com *exceptions*, que nada mais são do que classes específicas para cada tipo de erro, e que contém informações de quando e qual erro ocorreu, entre outros.
- O trecho de código em que uma exceção é esperada deve ser colocado dentro de um *try{}.* Caso a exceção esperada venha ocorrer, o seu tratamento é feito dentro do trecho *catch{}.* logo em seguida do *try{}.*

Exceptions

- Exemplo:

```
int x = (int) (Math.random() * 5);  
int y = (int) (Math.random() * 10);  
int [] z = new int[5];  
try {  
    System.out.println("y/x é " + (y/x));  
    System.out.println("y é " + y + " z[y] é " + z[y]);  
} catch (ArithmeticException e) {  
    System.out.println("Problema aritmético " + e);  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Erro no índice " + e);  
}
```

Exceptions



- Se uma exceção ocorrer e não for tratada em um determinado método, esta é passada ao método justamente anterior na pilha de execução, e assim por diante.

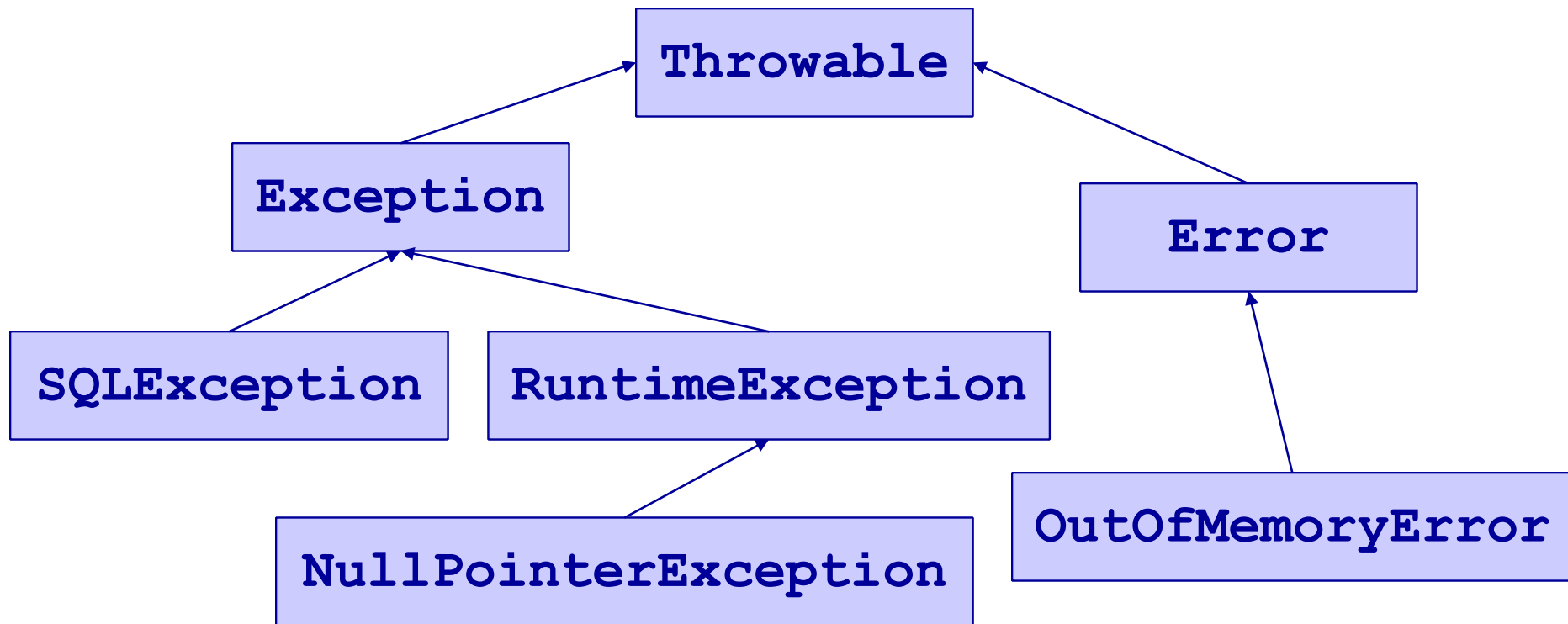
Exceptions

- Tratando múltiplas exceções e o bloco **finally**.

```
try {  
    // faz alguma coisa  
} catch (FileNotFoundException e) {  
    // trata o erro  
} catch (SQLException e) {  
    // trata o erro  
} catch (Exception e) {  
    // trata o erro genérico  
} finally {  
    /* finaliza o uso dos recursos (arquivos, transações,  
    etc) */  
}
```

Exceções e Hierarquia

- Todos os tipos de exceções e erros são filhos da classe **Throwable**



Lançando Exceções

- Vimos como podemos tratar exceções. Mas como lançar uma exceção?
- Exceções podem ser lançadas via o comando

```
throw new IOException("Arquivo não encontrado");
```

- Após a execução desta linha de código, o fluxo normal é interrompido e resumido somente no *catch* correspondente. Se não houver um *catch* explícito, quem trata o problema é o sistema operacional.

Checked Exceptions

- Entretanto, como podemos saber se um trecho de código é suscetível a lançamento de exceções?
- Em Java, qualquer método que pode lançar uma exceção deve declarar isto.

```
public void method1 throws SQLException {  
  
    // este método pode jogar uma SQLException  
  
}
```

- As exceções checadas são somente as subclasses de `Exception`, exceto `RuntimeException` e suas filhas. `Errors` também não são checadas.

Exceções - Vantagens

- Mantém separados os trechos de código que tratam condições anormais ou condições de erro
- Propagação das exceções pela pilha de execução, até que seja encontrado um tratador adequado para a exceção
- Tratamento genérico para tipos de exceção

Exceções - Boas Práticas

- Não tratar todas as exceções genericamente. Faça um tratamento adequado para cada tipo de exceção.
- Não lançar exceções genéricas. Crie novos tipos de exceções.
- Utilizar um mecanismo de *logging* para registrar a ocorrência da exceção.
- Utilize o bloco *finally* para finalizar o uso dos recursos

Exceções e Sobreescrita de Métodos

```
public class BaseClass {
    public void method() throws IOException{}
}

public class LegalOne extends BaseClass {
    public void method() throws IOException{}
}

public class LegalTwo extends BaseClass {
    public void method() {
    }
}

public class LegalThree extends BaseClass {
    public void method() throws EOFException, MalformedURLException {}
}

public class IllegalOne extends BaseClass {
    public void method() throws IOException, IllegalAccessException {}
}

public class IllegalTwo extends BaseClass {
    public void method() throws Exception {}
}
```